

**Analysis, Visualization, and Modeling of Credit Risk Using the Statlog (German Credit)
Dataset**

Data Mining

Teodora Veljanoska

Tampa, 2026

<i>Abstract.....</i>	<i>4</i>
<i>Introduction</i>	<i>4</i>
<i>Topic Description.....</i>	<i>4</i>
<i>Data Collection and Dataset Description.....</i>	<i>5</i>
<i>Data Structure.....</i>	<i>5</i>
<i>1. Preprocessing and Exploratory Data Analysis (EDA).....</i>	<i>6</i>
<i>1.1 Basic Statistical Analysis of Numerical Attributes.....</i>	<i>6</i>
<i>1.2 Analysis of Class Distribution (Class Imbalance)</i>	<i>7</i>
<i>1.3 Analysis of Categorical Attributes</i>	<i>7</i>
<i>1.4 Visual Exploratory Analysis.....</i>	<i>7</i>
<i>1.5 Checking for Missing Values.....</i>	<i>10</i>
<i>1.6 Encoding Categorical Attributes</i>	<i>10</i>
<i>1.7 Selection of the Most Important Predictors (Feature Selection).....</i>	<i>11</i>
<i>2. Classification.....</i>	<i>14</i>
<i>2.1 Splitting the Data into Training and Test Sets.....</i>	<i>14</i>
<i>2.2 Decision Tree Model</i>	<i>14</i>
<i>2.3 Random Forest Model</i>	<i>18</i>
<i>2.4 SVM Model.....</i>	<i>21</i>
<i>2.5 PR Analysis (Baseline Models).....</i>	<i>24</i>
<i>2.6 Cost-Sensitive Approach: class_weight='balanced'</i>	<i>25</i>
<i>2.7 Threshold Tuning (Focus: Final SVM_balanced Model).....</i>	<i>26</i>
<i>2.8 Brief Conclusion from Classification</i>	<i>29</i>
<i>2.9 Predictor Interpretation - Feature Importance</i>	<i>29</i>
<i>2.10 Controlled Feature Engineering Experiments (Brief Note)</i>	<i>30</i>
<i>3. Regression.....</i>	<i>31</i>
<i>3.1 Objective of the Regression Analysis.....</i>	<i>31</i>
<i>3.2 Linear Regression</i>	<i>31</i>
<i>3.3 Ridge Regression.....</i>	<i>32</i>
<i>3.4 SVR (Support Vector Regression).....</i>	<i>33</i>
<i>3.5 Model Comparison.....</i>	<i>35</i>
<i>3.6 Conclusion from Regression.....</i>	<i>35</i>
<i>4. Clustering</i>	<i>36</i>
<i>4.1 Objective of Clustering.....</i>	<i>36</i>

4.2 Model Selection.....	36
4.3 Data Preparation.....	37
4.4 Selection of an Appropriate Quality Metric.....	37
4.5 Final Selection of k	41
4.6 K-Means ($k=4$) - Results.....	41
4.7 Default Rate by Cluster.....	42
4.8 Cluster Profiles.....	43
4.9 Comparison of Different Values of k ($k=3$ and $k=5$).....	45
4.10 DBSCAN.....	48
4.11 Conclusion from Clustering.....	50
4.12 Most Important Predictors and Their Practical Meaning	50
5. Additional Techniques and Validation Experiments.....	52
5.1 Handling Imbalanced Classes (Cost-Sensitive Learning)	52
5.2 Dimensionality Reduction and Complexity Control.....	52
5.3 Feature Engineering Experiments (Controlled Improvements)	52
5.4 Note on Additional Ensemble Methods.....	54
6. Conclusion.....	55
References:	56

Abstract

This paper analyzes the Statlog (German Credit Data) dataset with the aim of developing a credit risk assessment model, i.e., predicting whether a client represents a risk of default. The data contains 1,000 credit applicants described through 20 attributes (categorical and numerical), where the target variable classifies clients as “good” and “bad” credit risks. The project combines supervised learning techniques (classification and modeling of `risk_score` as a continuous risk assessment) and unsupervised learning (clustering for customer segmentation). Due to the imbalanced class distribution and the different cost of errors in a banking context, in addition to ROC analysis, Precision-Recall metrics, cost-sensitive approaches (`class_weight`), and threshold selection (threshold tuning) are also used. In the final section, `risk_score` and clustering are used to rank and group clients with different default rates, enabling clearer application of credit policies, monitoring, and risk management.

<https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data>

Introduction

Credit risk is one of the most important factors in banking decision-making, because an incorrect assessment can directly lead to financial losses. Modern credit approval systems increasingly use machine learning methods to assess whether there is a risk that an applicant will become a default case. However, such problems are usually characterized by imbalanced classes and different “costs” of errors, especially when a risky client is classified as safe.

This project uses the well-known benchmark dataset Statlog (German Credit Data), which describes credit applicants through financial and demographic attributes. The goal is to build a model that will assist in credit risk assessment through: (1) binary classification of good/bad clients, (2) calculation of a continuous `risk_score` for ranking, and (3) customer segmentation through clustering, where the target is used only for interpretation of the resulting segments.

Topic Description

The project aims to provide a complete analytical pipeline for credit risk assessment: from exploratory analysis and preprocessing, through model training and evaluation, to extracting practical knowledge about the most important predictors and segmenting clients. The data contains a mixture of categorical and numerical attributes, so one-hot encoding is applied to enable the use of machine learning models.

Due to the imbalanced class distribution and the banking cost of errors, in addition to ROC, Precision-Recall analysis is also used, as well as a cost-sensitive approach through `class_weight` and threshold selection (threshold tuning). Additionally, a `risk_score` is constructed as a continuous

assessment (probability of default), while clustering produces customer segments that are subsequently validated through default rate and risk_score. This approach allows the results to be not only “accurate,” but also applicable in the context of credit policies, monitoring, and risk-based ranking. The goal is not only to maximize accuracy, but to build an interpretable and practically applicable model that will enable better credit risk management.

Data Collection and Dataset Description

Data Source

This project uses the German Credit (StatLog) dataset, originally prepared by Prof. Dr. Hans Hofmann (University of Hamburg). This dataset represents a standard and widely used benchmark dataset in the field of credit risk analysis and machine learning. The data contains information about credit applicants and their final assessment from the perspective of creditworthiness, i.e., whether the client is considered a “good” or “bad” credit risk.

Data Structure

The dataset contains:

- 1,000 instances (rows) - each row represents one credit applicant
- 20 attributes (predictors) - describing financial, demographic, and credit characteristics

The attributes include information such as:

- credit amount
- credit duration
- credit history
- employment status
- savings
- age
- housing type
- number of existing credits

Some attributes are numerical and some are categorical, where categorical values are encoded using symbolic labels (e.g., A11, A12, A13). Therefore, before applying machine learning algorithms, additional preprocessing and encoding of these variables are required.

Definition of the Target Variable

The target variable is default, which in the original file has the following values:

- 1 = good credit (low risk)
- 2 = bad credit (high risk / risk of default)

For classification purposes, the target is transformed into binary form:

- 0 = good credit
- 1 = bad credit (default / risk of default)

This transformation enables simpler application of binary classification algorithms.

Additionally, we examine the distribution of the classes and observe that the classes are imbalanced:

- approximately 70% good clients
- approximately 30% bad clients

Due to the imbalance ($\approx 70\%$ good / $\approx 30\%$ bad), accuracy alone is not sufficient. Therefore, the evaluation uses ROC-AUC and Precision/Recall, with a focus on Recall for “bad” clients, while additionally applying a cost-sensitive approach and threshold selection (threshold tuning).

1. Preprocessing and Exploratory Data Analysis (EDA)

1.1 Basic Statistical Analysis of Numerical Attributes

For a basic overview of the data, we use descriptive statistics with `describe()` (Pandas), which provides the main measures of central tendency and variability (mean, std, min/max, and quartiles). This helps us quickly assess whether there are anomalies and whether problems are expected during modeling.

Key findings:

- amount has a right-skewed distribution and the presence of pronounced outliers (the maximum is significantly larger than the average)
- duration varies over a wide range (up to 72 months), which may affect risk and model performance

This step serves as the basis for visual analysis and further preprocessing.

1.2 Analysis of Class Distribution (Class Imbalance)

Using the `value_counts()` function, we analyze the distribution of the target variable.

The obtained results show:

- 700 clients are classified as “good”
- 300 clients are classified as “bad”

This confirms class imbalance ($\approx 70/30$), which is later addressed through PR analysis, `class_weight`, and threshold tuning.

1.3 Analysis of Categorical Attributes

For categorical variables, we check the number of categories in order to assess their complexity and potential informativeness. The results show:

- `purpose` has 10 categories (greater diversity)
- `credit_history`, `employment`, and `savings` each have 5 categories
- `telephone` and `foreign_worker` have 2 categories (simpler binary attributes)

This is important because more categories mean more one-hot columns after encoding, which can increase dimensionality and affect the models.

1.4 Visual Exploratory Analysis

In addition to numerical statistical analysis, several key visualizations are created in order to obtain a more intuitive understanding of the data distribution, potential anomalies, and client characteristics. Instead of a large number of graphs, a small number of informative visualizations are used that provide direct insight into the structure of the dataset.

1.4.1 Class Distribution

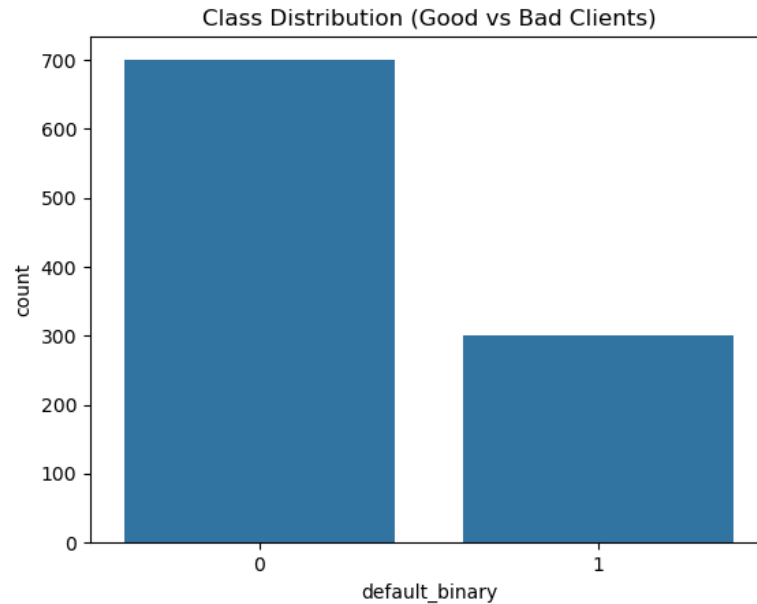


Figure 1. Class distribution (good versus bad clients)

Interpretation: The dataset is imbalanced ($\approx 70/30$), so the results will be evaluated using PR/ROC metrics and the threshold will be optimized to reduce the number of FN cases.

1.4.2 Age Distribution

The ages are most commonly between 25-40, with no anomalies. The data is realistic and no additional processing of this variable is required.

1.4.3 Distribution of Credit Amount

The distribution of the credit amount is clearly right-skewed, with most clients taking smaller loans, while a small number of clients have significantly larger amounts. This asymmetry may affect the performance of certain models and indicates a possible need for normalization or transformation of the data.

1.4.4 Detection of Extreme Values (Outliers)

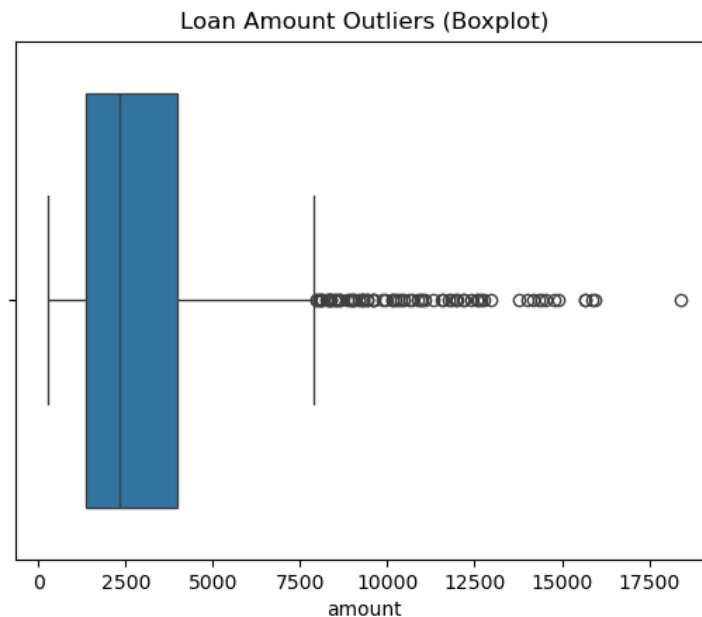


Figure 2. Detection of extreme values (outliers) in credit amount - boxplot

The boxplot further confirms the presence of a large number of extreme values in the amount attribute. These values differ significantly from the majority of records and may negatively affect models, especially regression methods.

Although extreme values were detected in amount, they were not removed because they represent realistic high-credit cases. Instead, their influence is examined through a controlled log transformation in the experimental section.

1.4.5 Conclusion from the Visual Analysis

The visualizations enabled:

- detection of class imbalance
- better understanding of the demographic structure
- identification of asymmetric distributions
- detection of extreme values

These findings will be used during the preprocessing and modeling phase.

1.5 Checking for Missing Values

In order to determine whether the dataset contains missing values, we perform a check using the `isnull().sum()` function from the Pandas library, which calculates the number of empty values for each attribute.

The results show that none of the attributes contains missing values, meaning that all 1,000 records are complete. This means that no additional processing such as elimination or imputation (mean/median filling) is required, and we can proceed directly to the next preprocessing steps.

1.6 Encoding Categorical Attributes

A large portion of the attributes in the dataset are categorical and contain symbolic values (e.g., A11, A12, A13, as well as different types of credit purpose, employment, housing, etc.). These values cannot be directly used by machine learning algorithms because the models operate exclusively on numerical data.

If the categories were converted into integers (e.g., 1, 2, 3), the model would assume that an order or ranking exists between them, which is not logically justified.

Therefore, we apply the One-Hot Encoding technique using the `get_dummies()` function from the Pandas library. With this technique, each category is transformed into a separate binary (0/1) variable.

For example, instead of one categorical purpose column, it is transformed into multiple binary columns such as:

- `purpose_car`
- `purpose_business`
- `purpose_furniture`
- etc.

In this way:

- categorical data is converted into numerical data
- false ordering between categories is avoided
- proper application of classification, regression, and clustering algorithms is enabled

After encoding, the number of attributes increased from 21 to 50, which is expected because each category is represented as a separate variable. This transformation prepares the dataset for further modeling.

1.7 Selection of the Most Important Predictors (Feature Selection)

In order for the project to focus on extracting real knowledge from the data, rather than merely applying algorithms, at this stage we analyze the relationship between each predictor and the target variable `default_binary` (risk of default). After encoding the categorical attributes (One-Hot Encoding), all variables are in numerical form (0/1 or continuous values), allowing us to calculate Pearson correlation between each predictor and the target.

Correlation has values in the interval $[-1, 1]$, where:

- a positive value -> as the predictor increases/is present, the probability of default increases
- a negative value -> as the predictor increases/is present, the probability of default decreases
- values close to 0 -> weak or insignificant relationship

This allows us to identify early the factors that contribute most to credit risk.

1.7.1 Predictors Associated with Higher Default Risk (Positive Correlation)

The results show that the strongest positive association with default is found in:

- `duration` - longer credit repayment period
- `amount` - larger credit amount
- `installment_rate` - higher percentage of the monthly installment relative to income
- `credit_history_A31` - certain types of previous credit history
- `employment_A72` - short employment period

These factors indicate that clients with higher debt, longer repayment periods, and a less stable financial situation have an increased probability of default.

1.7.2 Predictors Associated with Lower Default Risk (Negative Correlation)

The analysis shows that several attributes have a negative correlation with the target variable, meaning that their presence or higher value is associated with a reduced risk of default.

The strongest negative associations are:

- `status_A14` - no current account (specific financial profile)
- `credit_history_A34` - critical account / existing credits, but historically regular repayment
- `housing_A152` - own housing
- `age` - older clients have lower risk
- `savings_A64` - higher savings

These results are financially logical because more stable, experienced, and financially secure clients have a lower probability of default.

1.7.3 Reducing the Number of Predictors (Feature Selection)

After applying One-Hot Encoding, the number of attributes increases from 21 to more than 50 variables. Using all of these predictors may lead to:

- increased model complexity
- more difficult interpretation
- higher risk of overfitting
- weaker communication of results

Therefore, at this stage we intentionally reduce the number of predictors and focus only on those that provide the greatest informational value.

We calculate the Pearson correlation coefficient between each predictor and the default_binary target variable and retain the attributes with the strongest positive or negative association, combined with financial and domain logic.

The selected 13 predictors are:

- duration
- amount
- installment_rate
- age
- residence_time
- existing_credits
- people_liable
- credit_history_A31
- credit_history_A34
- housing_A152
- savings_A64
- employment_A72
- status_A14

These predictors are used further because:

- statistically, they show the strongest association with default risk
- they have a clear and logical financial interpretation
- they enable simpler and more understandable models
- they reduce the risk of overfitting
- they enable easier extraction of conclusions and communication of results

With this approach, modeling is based on the most important factors that actually influence credit risk, rather than on a large number of weakly informative attributes.

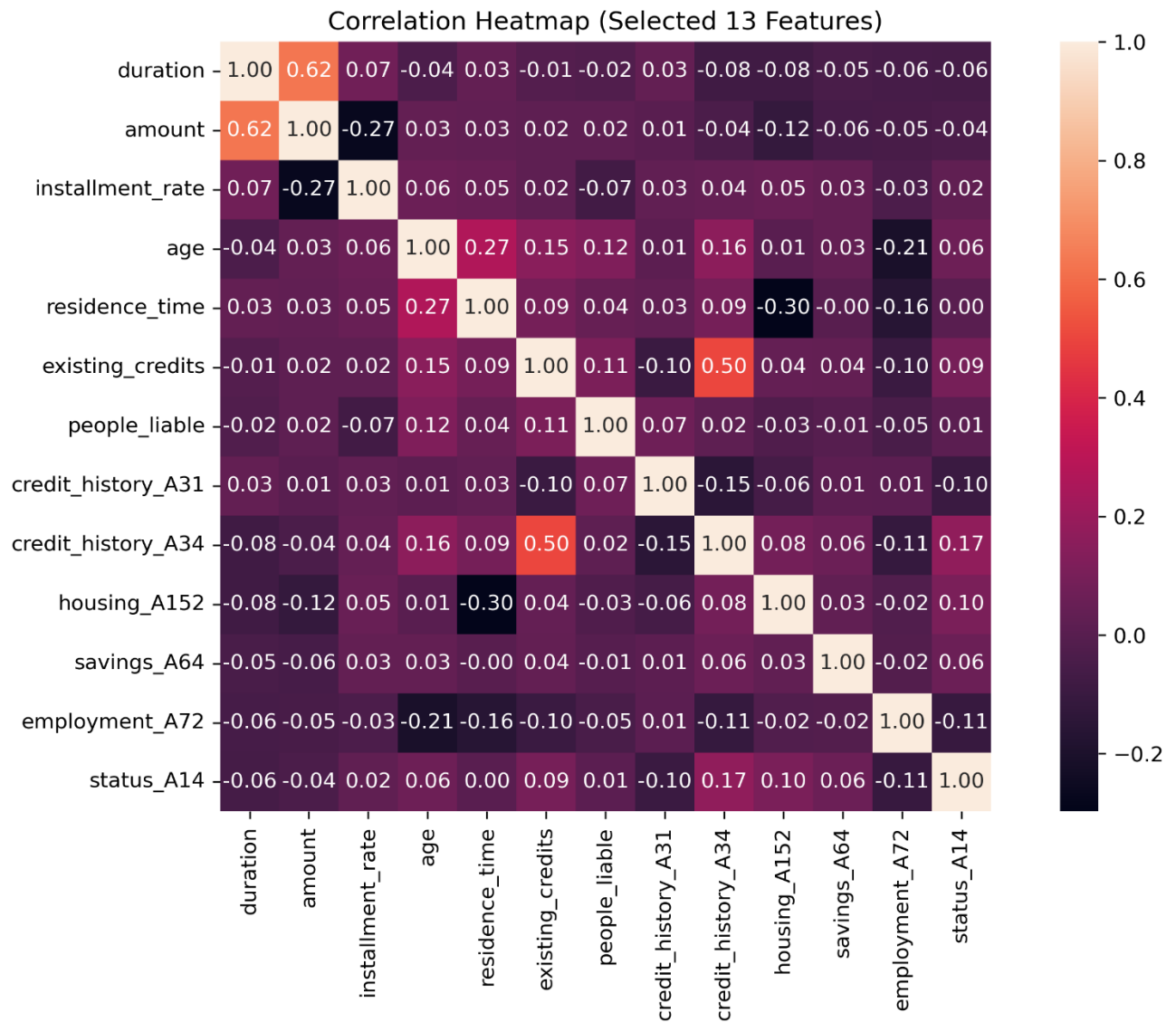


Figure 3. Correlation matrix of the selected 13 predictors

The correlation matrix shows that there is no pronounced multicollinearity among the selected predictors. Although some variables have a moderate relationship (for example, amount and duration), the values are within an acceptable range. This confirms that the models are not significantly affected by strong interdependence between the attributes.

2. Classification

2.1 Splitting the Data into Training and Test Sets

After completing preprocessing and selecting the 13 most important predictors, we proceed to prepare the data for modeling.

At this stage, the dataset is divided into two subsets:

- Training set (80%) - used for training the models
- Test set (20%) - used for independent evaluation of performance

The split is performed using the `train_test_split()` function from the `scikit-learn` library.

Additionally, we use the `stratify=y` option, which preserves the same class distribution ($\approx 70\%$ good / 30% bad) in both the training and test sets. This is especially important due to the presence of imbalanced classes.

After the split we obtain:

- 800 records for training
- 200 records for testing
- 13 selected predictors

This approach enables realistic evaluation of the models on data that was not previously seen by the algorithm, reducing the risk of overfitting and providing a more objective assessment of their accuracy.

2.2 Decision Tree Model

At this stage, we begin applying the first classification model - Decision Tree.

Decision Tree is a simple and easily interpretable model that works as a series of logical rules of the if-else type. The model automatically selects the most informative predictors and builds a hierarchy of decisions through which each client is classified as a “good” or “bad” credit risk.

This type of model is particularly suitable for financial problems because it enables:

- clear interpretation
- visual understanding of decisions
- easy analysis of factors affecting risk

The model is created with:

- `DecisionTreeClassifier(max_depth=4, random_state=42)`

The `max_depth = 4` parameter limits the tree to a maximum of four decision levels in order to avoid excessive complexity and overfitting and to obtain a more stable model.

The model is trained on the training set using:

- `dt.fit(X_train, y_train)`

Then classes are predicted for the test set using:

- `y_pred_dt = dt.predict(X_test)`

Evaluation is performed using:

- `confusion_matrix()`
- `classification_report()`
- `roc_curve()` and `roc_auc_score()`

2.2.1 Evaluation Results

Confusion Matrix

	Predicted: Good	Predicted: Bad
Actual: Good	116	24
Actual: Bad	34	26

Table 1. Confusion Matrix for the Decision Tree model (test set)

From the table we can see that:

- 116 good clients were correctly classified
- 24 good clients were incorrectly labeled as bad
- 26 bad clients were correctly detected
- 34 bad clients were incorrectly classified as good (a critical banking error)

2.2.2 Performance Metrics

The metrics are calculated automatically using `classification_report()`.

Accuracy

Correct predictions = $116 + 26 = 142$

Total cases = 200

Accuracy = $142 / 200 = 0.71 \rightarrow 71\%$

The model is correct in 71% of all cases.

Precision (for bad clients)

Precision = $26 / (26 + 24)$

Precision = $26 / 50 = 0.52 \rightarrow 52\%$

When the model says “bad client,” it is correct in approximately half of the cases.

Recall (for bad clients)

Recall = $26 / (26 + 34)$

Recall = $26 / 60 = 0.43 \rightarrow 43\%$

The model catches only 43% of all actually risky clients.

This is the most important metric in a banking context because a missed bad client can cause financial loss.

F1-score

F1 = $2 * (0.52 * 0.43) / (0.52 + 0.43)$

F1 $\approx 0.47 \rightarrow 47\%$

F1-score represents a compromise between precision and recall and indicates moderate performance.

2.2.3 ROC Analysis

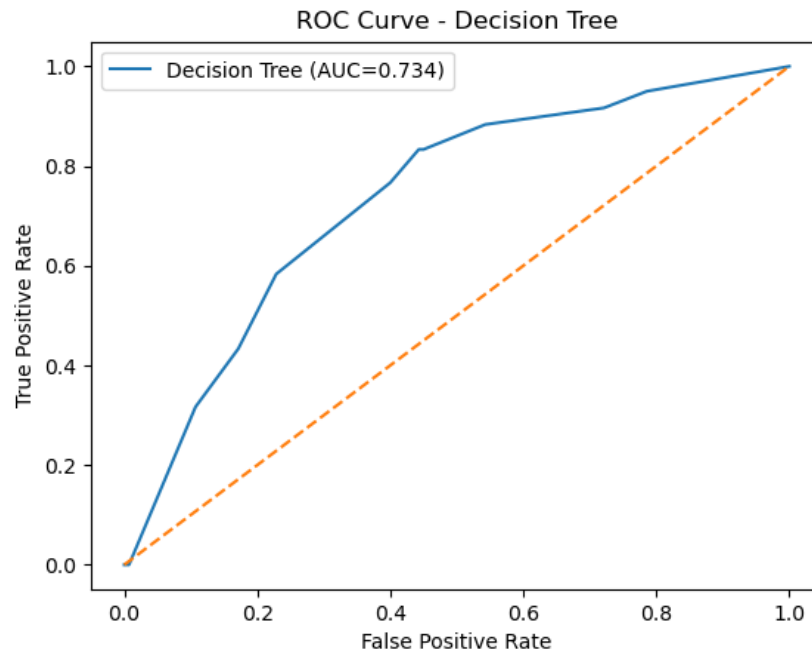


Figure 4. ROC curve for Decision Tree model

ROC analysis is used to evaluate how well the model separates good and bad clients at different thresholds.

The ROC curve is obtained using:

- `predict_proba()`
- `roc_curve()`
- `roc_auc_score()`

$AUC \approx 0.73 \rightarrow 73\%$

This means that the model has a 73% probability of correctly distinguishing a randomly selected good client from a randomly selected bad client.

The value is better than random guessing (50%), but it is still not high enough for reliable banking application.

2.2.4 Conclusion

The Decision Tree model:

- is simple and easily interpretable
- provides solid initial accuracy
- but has low recall for the risky class

This means that it misses a large number of bad clients, which represents a risk for the bank.

Therefore, this model is used as a baseline (initial comparison point), while more advanced models such as Random Forest are applied in the following steps with the goal of improving credit risk detection.

2.3 Random Forest Model

We move on to the second classification model - Random Forest.

Random Forest is an ensemble method that builds multiple Decision Tree trees using different “subsamples” of the data and different subsets of predictors, and then makes a final decision through voting (majority vote). This usually produces more stable and general results than a single tree because variance and the risk of overfitting are reduced.

The model is trained on the training set and evaluated on the test set. The following functions are used for evaluation:

```
confusion_matrix(y_test, y_pred_rf)
```

```
classification_report(y_test, y_pred_rf)
```

ROC analysis using `roc_curve()` and `roc_auc_score()` (using `predict_proba()`)

2.3.1 Evaluation Results

Confusion Matrix

	Predicted: Good	Predicted: Bad
Actual: Good	123	17
Actual: Bad	34	26

Table 2. Confusion Matrix for Random Forest model (test set)

From the table we observe:

123 good clients were correctly classified.

17 good clients were incorrectly labeled as bad.

26 bad clients were correctly detected.

34 bad clients were incorrectly classified as good (FN).

2.3.2 Performance Metrics

The metrics are calculated automatically using the `classification_report()` function from the `scikit-learn` library.

Accuracy

Correct predictions = $123 + 26 = 149$

Total cases = 200

Accuracy = $149 / 200 = 0.745 \rightarrow 74\%$

The model is correct in approximately 74% of all cases, representing an improvement over the Decision Tree model.

Precision (for bad clients)

Precision = $26 / (26 + 17)$

Precision = $26 / 43 = 0.60 \rightarrow 60\%$

When the model predicts a “bad client,” it is correct in approximately 60% of cases.

This represents an improvement over Decision Tree, meaning that there are fewer false alarms.

Recall (for bad clients)

Recall = $26 / (26 + 34)$

Recall = $26 / 60 = 0.43 \rightarrow 43\%$

The model catches only 43% of all actually risky clients.

This means that 34 bad clients are missed, representing a critical error in a banking context.

Despite the improvement in accuracy and precision, the ability to detect risk remains limited.

F1-score

$$F1 = 2 * (0.60 * 0.43) / (0.60 + 0.43)$$

$$F1 \approx 0.50 \rightarrow 50\%$$

F1-score represents a compromise between precision and recall.

A value of approximately 50% indicates moderate performance in recognizing risky clients.

2.3.3 ROC Analysis

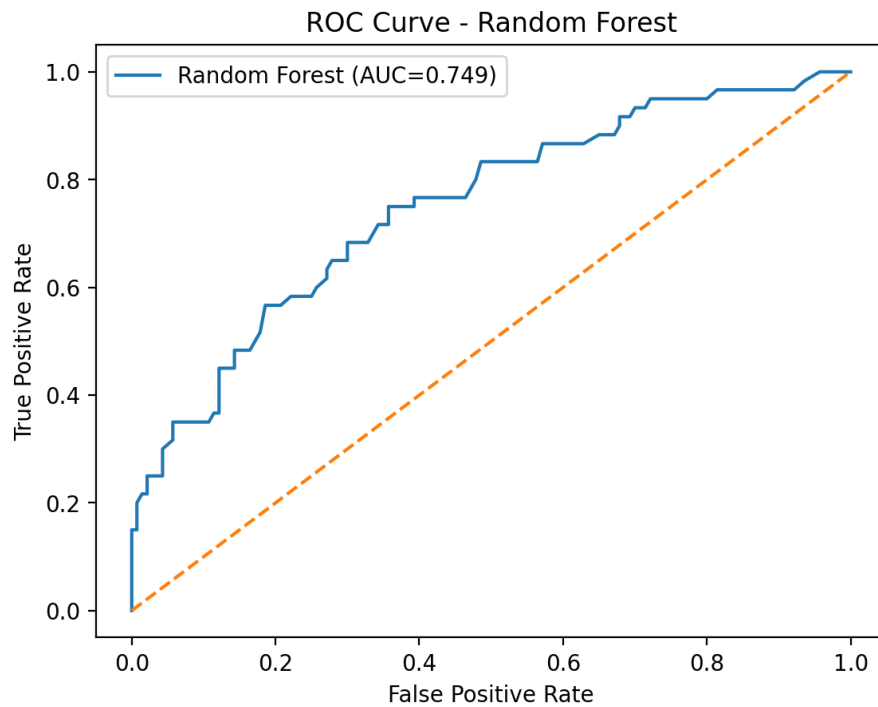


Figure 5. ROC curve for Random Forest model

The ROC curve shows the model's ability to distinguish good and bad clients at different decision thresholds. The AUC value is calculated using `roc_auc_score`.

$$\text{ROC-AUC} \approx 0.749$$

This means that the model has approximately a 74.9% probability of assigning a higher risk score to a randomly selected “bad” client than to a randomly selected “good” client.

The value is better than that of Decision Tree (≈ 0.734), but there is still room for improvement, particularly in detecting the risky class.

2.3.4 Conclusion

Random Forest:

- provides better accuracy than Decision Tree ($\approx 74\%$ versus $\approx 71\%$)
- has a slightly better ROC-AUC (≈ 0.749)
- but recall for the bad class remains $\approx 43\%$, which is critical in credit risk

Therefore, in the next step a third model will be applied that often provides better class separation (for example, SVM), with the goal of improving detection of risky clients.

2.4 SVM Model

We move on to the third classification model - SVM (Support Vector Machine).

SVM is a model that attempts to find the best boundary (separating line/hyperplane) that separates “good” and “bad” clients while maximizing the margin between the classes.

This model often performs well when classes can be separated by a clear boundary, but it is sensitive to scaling, so it is usually used together with standardization.

It is suitable for tabular data with standardization and is also used because of its ability to obtain a risk_score through predict_proba / decision_function.

The model is trained on the training set and evaluated on the test set. The following functions are used for evaluation:

- confusion_matrix(y_test, y_pred_svm)
- classification_report(y_test, y_pred_svm)
- ROC analysis using predict_proba() / decision_function(), roc_curve() and roc_auc_score()

2.4.1 Evaluation Results

Confusion Matrix

	Predicted: Good	Predicted: Bad
Actual: Good	131	9
Actual: Bad	34	26

Table 3. Confusion Matrix for SVM model (test set)

From the table we observe:

- 131 good clients were correctly classified
- 9 good clients were incorrectly labeled as bad
- 26 bad clients were correctly detected
- 34 bad clients were incorrectly classified as good (FN)

2.4.2 Performance Metrics

The metrics are calculated automatically using `classification_report()`.

Accuracy

Correct predictions = $131 + 26 = 157$
Total cases = 200

Accuracy = $157 / 200 = 0.785 \rightarrow 79\%$

The model is correct in 79% of all cases.

Precision (for bad clients)

Precision = $26 / (26 + 9)$

Precision = $26 / 35 = 0.743 \rightarrow 74\%$

When the model says “bad client,” it is correct in approximately 74% of cases (few false alarms).

Recall (for bad clients)

Recall = $26 / (26 + 34)$

Recall = $26 / 60 = 0.433 \rightarrow 43\%$

The model catches 43% of all actually risky clients, meaning that 34 bad clients are missed (risk for the bank).

(Note: Recall was $\approx 43\%$ for both DT and RF as well, so there is no improvement here in detecting risky clients.)

F1-score (for bad clients)

$$F1 = 2 * (0.74 * 0.43) / (0.74 + 0.43)$$

$$F1 \approx 0.55 \rightarrow 55\%$$

F1-score is a compromise between precision and recall and is better here than for DT/RF, mainly due to the higher precision.

2.4.3 ROC Analysis

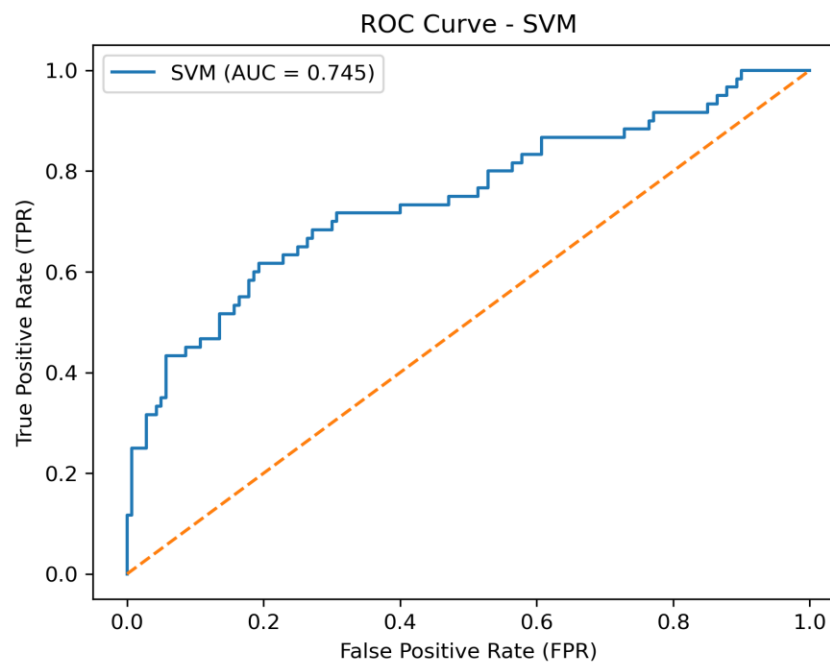


Figure 6. ROC curve for SVM model

The ROC curve shows the model's ability to distinguish good and bad clients at different thresholds, while AUC summarizes this ability in a single value.

- ROC-AUC $\approx 0.745 \rightarrow 74.5\%$

This means that the model has approximately a 74.5% probability of assigning a “higher risk score” to a randomly selected bad client than to a randomly selected good client.

2.4.4 Conclusion

SVM:

- provides the highest accuracy of the three baseline models ($\approx 79\%$)
- has the highest precision for the bad class ($\approx 74\%$) and the best F1 ($\approx 55\%$)
- but recall for the bad class remains $\approx 43\%$ -> it misses many risky clients

Therefore, in the next phase we analyze why recall remains low (imbalanced data and decision threshold) and apply improvements such as `class_weight='balanced'`, PR curve, and threshold tuning.

2.5 PR Analysis (Baseline Models)

Because the classes are imbalanced ($\approx 70/30$), in addition to ROC we use a Precision-Recall (PR) curve, which is more informative when the main focus is the positive class (bad clients).

The PR curve and PR-AUC (Average Precision) are obtained using:

- `precision_recall_curve()`
- `average_precision_score()`

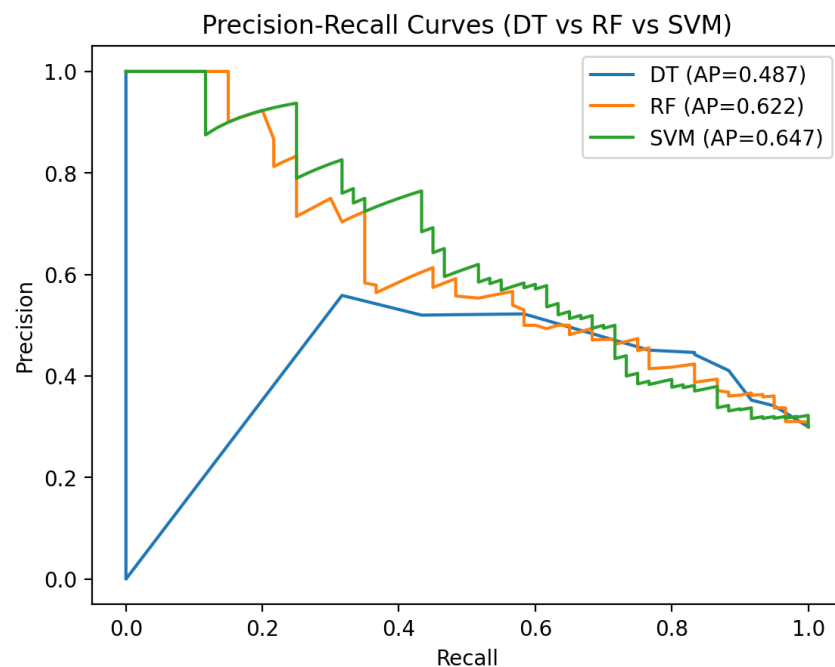


Figure 7. Precision-Recall curves (DT vs RF vs SVM)

PR-AUC (AP):

- DT: ≈ 0.487
- RF: ≈ 0.622
- SVM: ≈ 0.647

Interpretation: RF and SVM provide better ranking of “bad” clients according to PR-AUC, although recall at the default threshold remains low.

2.6 Cost-Sensitive Approach: `class_weight='balanced'`

Because the dataset is imbalanced and the cost of misclassifying risky clients is potentially greater than the cost of additional manual review, a cost-sensitive approach was applied using `class_weight='balanced'`. Threshold tuning was then used to examine the trade-off between precision and recall for the risky class. The original UCI dataset also provides a cost matrix, but this project focuses on the class-weighting and threshold-based approach rather than directly optimizing the original cost matrix.

Evaluation is performed using:

- `confusion_matrix()`
- `classification_report()`
- `roc_auc_score()`
- `average_precision_score()` (PR-AUC)

Key results (balanced):

DT_balanced

$\text{Recall_bad}(1) \approx 0.77$, $\text{Precision_bad}(1) \approx 0.46$, $\text{Accuracy} \approx 0.66$

-> raises recall the most, but creates many false alarms.

RF_balanced

$\text{Recall_bad}(1) \approx 0.38$, $\text{Precision_bad}(1) \approx 0.64$, $\text{Accuracy} \approx 0.75$

-> does not improve recall sufficiently.

SVM_balanced

$\text{Recall_bad}(1) \approx 0.78$, $\text{Precision_bad}(1) \approx 0.48$, $\text{Accuracy} \approx 0.69$

-> significant improvement in recall with a moderate cost in precision.

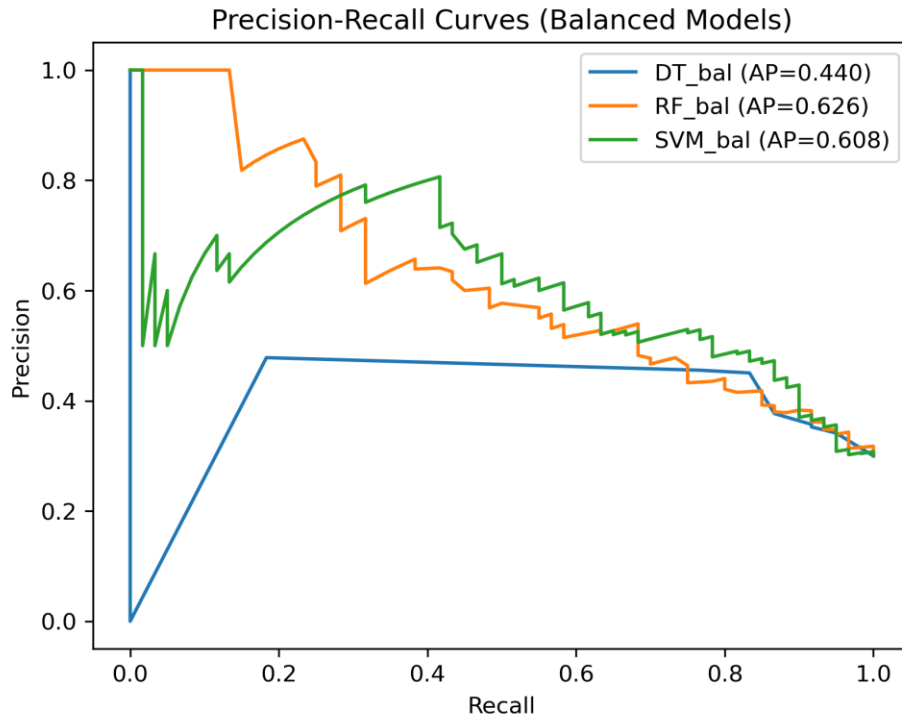


Figure 8. Precision-Recall curves (Balanced models)

PR-AUC (AP):

- DT_balanced: ≈ 0.440
- RF_balanced: ≈ 0.626
- SVM_balanced: ≈ 0.608

2.7 Threshold Tuning (Focus: Final SVM_balanced Model)

The standard threshold is 0.50, but in a banking context a lower threshold is often selected in order to capture more risky clients.

Therefore, we perform threshold tuning using `predict_proba()` and test thresholds from 0.50 -> 0.20.

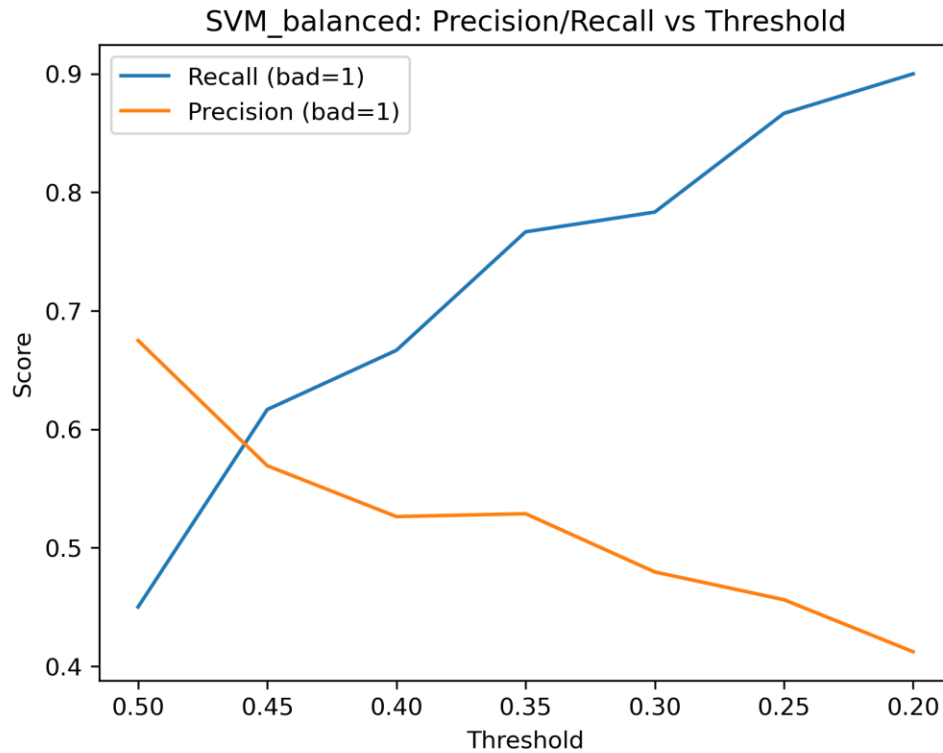


Figure 9. SVM_balanced: Precision/Recall vs Threshold

Results (excerpt):

- threshold 0.50 -> precision ≈ 0.68 , recall ≈ 0.45
- threshold 0.35 -> precision ≈ 0.53 , recall ≈ 0.77 (best F1 for bad)
- threshold 0.25 -> precision ≈ 0.46 , recall ≈ 0.87
- threshold 0.20 -> precision ≈ 0.41 , recall ≈ 0.90

Interpretation: lowering the threshold increases recall (we capture more bad clients), but decreases precision (more false alarms).

As a practical compromise, **SVM_balanced with threshold ≈ 0.35** provides high detection (recall) and the best balance according to F1.

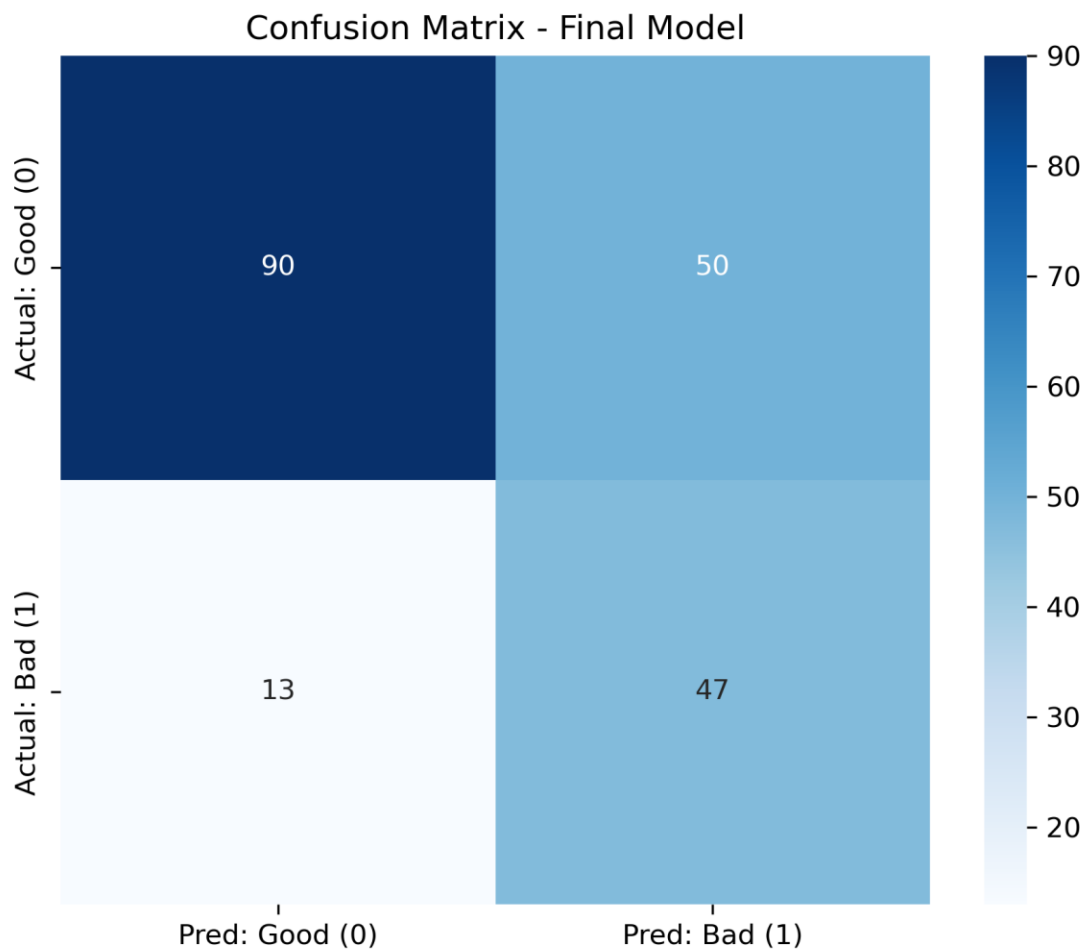


Figure 10. Confusion Matrix - Final Model (SVM_balanced, optimal threshold)

The confusion matrix shows that the final model significantly improves the detection of risky clients compared with the baseline models. The number of missed bad clients (false negatives) is reduced, which is crucial in a banking context. Although the number of additional checks (false positives) increases, this trade-off is justified by the reduced credit risk.

Additionally, the possibility of applying more advanced ensemble methods (for example, Gradient Boosting) was considered. However, given the moderate size of the dataset (1,000 instances) and the already stable performance ($\text{ROC-AUC} \approx 0.75$), further increasing complexity was not expected to bring a significant improvement in generalization. Therefore, the focus was placed on interpretability and cost-sensitive optimization, which are more critical in a banking context.

2.8 Brief Conclusion from Classification

The baseline models have recall ≈ 0.43 for bad clients, which is insufficient for credit risk. With `class_weight='balanced'` and threshold tuning, recall can be increased to ≈ 0.77 - 0.90 , which is more aligned with banking logic (it is better to have more checks than to miss a risky client).

In the next step, interpretation/knowledge extraction (feature importance) will be performed, followed by Regression and Clustering.

2.9 Predictor Interpretation - Feature Importance

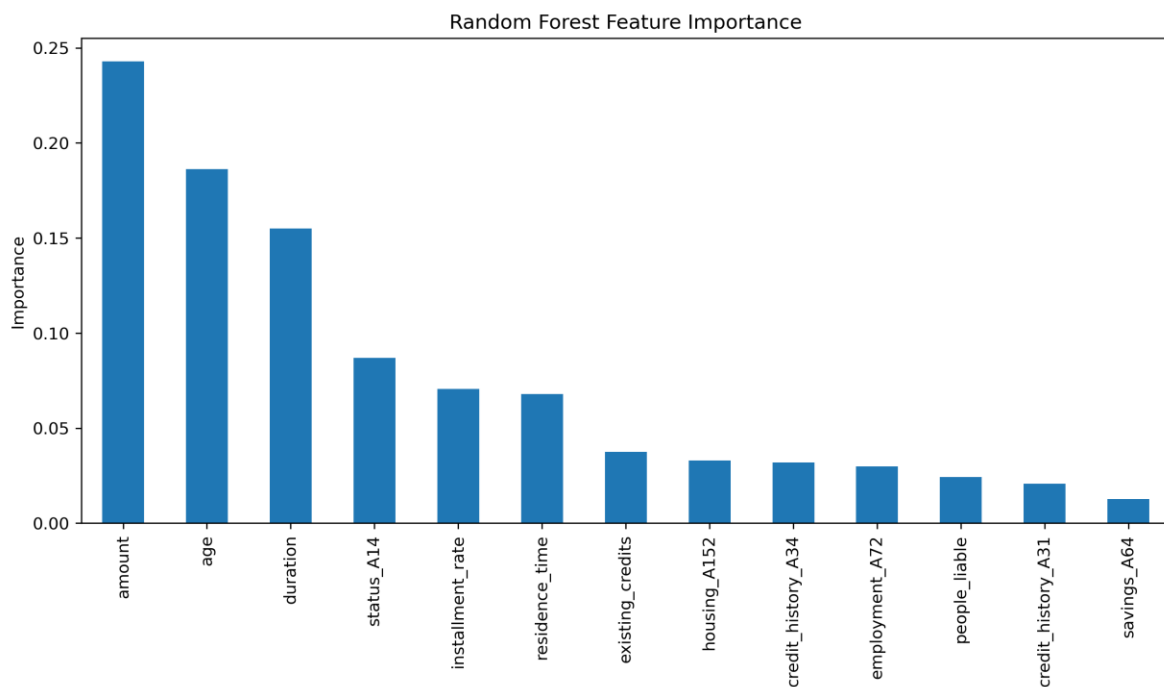


Figure 11. Random Forest - Feature Importance

The graph shows the relative importance of each of the 13 selected predictors in the Random Forest model. amount, age, and duration have the greatest influence, further confirming their central role in credit risk assessment. Smaller, but still relevant contributions come from credit_history, employment, and savings. This analysis strengthens the interpretability of the model.

2.10 Controlled Feature Engineering Experiments (Brief Note)

In addition to model selection and optimization through `class_weight` and threshold tuning, a limited form of feature engineering was also tested in order to determine whether simple transformations could provide additional discriminatory power without changing the basic pipeline.

The detailed results of the experiments ($\log_{10}(\text{amount})$ and `monthly_burden = amount/duration`), together with their impact on PR-AUC/ROC-AUC and clustering segmentation, are presented in Section 6.

	importance
amount	0.242920
age	0.186307
duration	0.155088
status_A14	0.086934
installment_rate	0.070598
residence_time	0.067996
existing_credits	0.037625
housing_A152	0.032893
credit_history_A34	0.031905
employment_A72	0.029918
people_liable	0.024393
credit_history_A31	0.020810
savings_A64	0.012615

Table 4. Ranked importance of predictors (Random Forest)

3. Regression

3.1 Objective of the Regression Analysis

In addition to binary classification (good/bad client), at this stage we also apply regression models with the goal of:

- predicting a continuous risk_score (probability of default)
- enabling ranking of clients according to risk
- obtaining better interpretation of the influence of predictors

Unlike classification, which provides a final decision (0/1), regression enables a more fine-grained assessment of risk, which is useful in real banking systems (credit scoring).

It is important to emphasize that risk_score does not represent an actual financial loss, but rather a proxy probability of default obtained from a classification model (predict_proba). Therefore, regression analysis is used for ranking and interpretation, rather than as a direct forecast of loss.

The evaluation is performed using:

- mean_squared_error()
- $RMSE = \sqrt{MSE}$
- r2_score()

where:

- lower MSE/RMSE -> better model
- R^2 close to 1 -> good explanation of variance

3.2 Linear Regression

Linear Regression represents the simplest regression model and assumes a linear relationship between the predictors and risk.

It is trained using:

- `LinearRegression().fit(X_train, y_train)`

and makes predictions using:

- `model.predict(X_test)`

Results:

- $MSE \approx 0.0075$
- $RMSE \approx 0.0867$
- $R^2 \approx 0.762$

This means that the model explains approximately 76% of the variation in risk, representing a very good approximation.

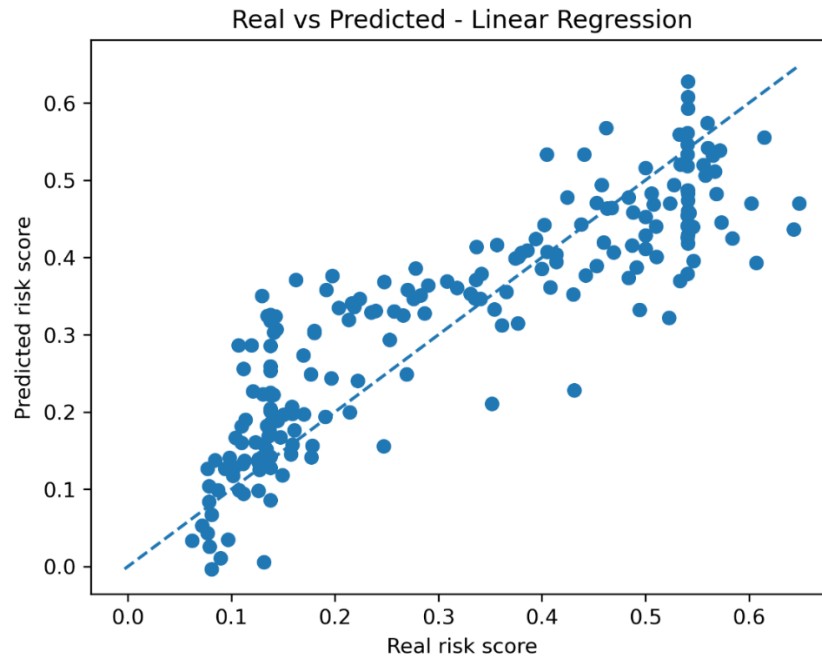


Figure 12. Real vs Predicted - Linear Regression

The points are distributed close to the diagonal, indicating good agreement between the actual and predicted values. This confirms that the relationship between the selected 13 predictors and risk_score can largely be described by a linear function.

3.3 Ridge Regression

Ridge Regression is an extension of Linear Regression with L2 regularization, which:

- reduces overfitting
- stabilizes coefficients
- improves generalization

Results:

- $MSE \approx 0.00753$
- $RMSE \approx 0.0868$
- $R^2 \approx 0.761$

The performance is almost identical to Linear Regression, showing that:

- there is no pronounced overfitting problem
- there is no strong multicollinearity requiring regularization
- the linear structure is stable

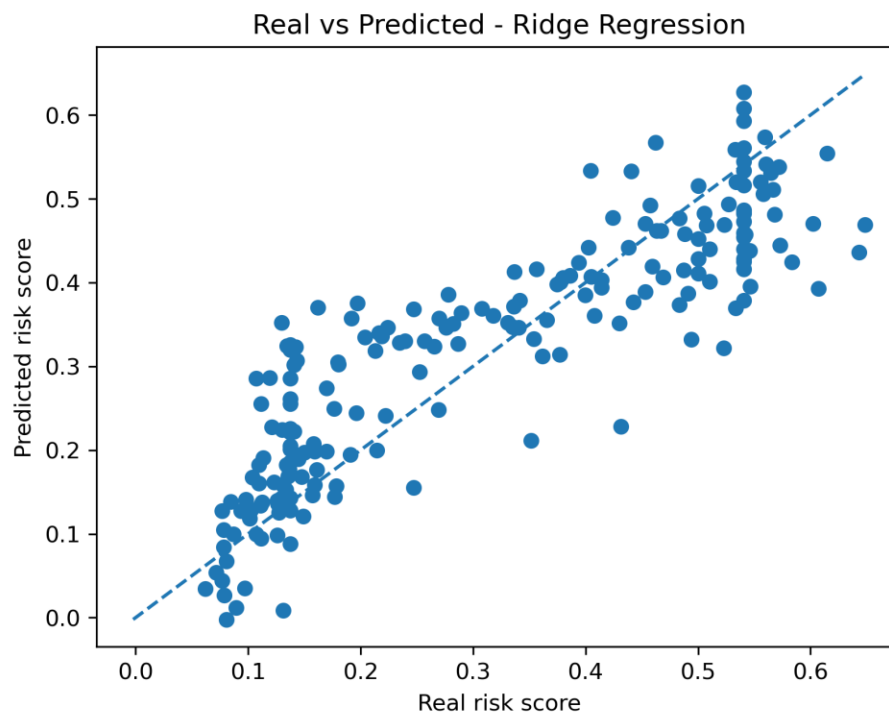


Figure 13. Real vs Predicted - Ridge Regression

3.4 SVR (Support Vector Regression)

Additionally, a nonlinear regression model - SVR with an RBF kernel - was tested.

The goal was to determine whether a more complex nonlinear function could better model risk_score.

Results:

- $MSE \approx 0.2267$
- $RMSE \approx 0.476$
- $R^2 \approx -0.08$

The negative R^2 value indicates that the model is worse than simply predicting the average.

It is important to emphasize that `risk_score` is a continuous proxy score in the interval $[0,1]$, rather than a binary variable. However, its distribution is concentrated and strongly related to a linear combination of predictors (because it originates from a classification model).

In this setup, the nonlinear SVR with standard parameters fails to learn stable generalization, resulting in significantly worse performance.

This result further indicates that the relationship between the selected predictors and `risk_score` is largely linear, so a complex nonlinear model does not provide additional value.

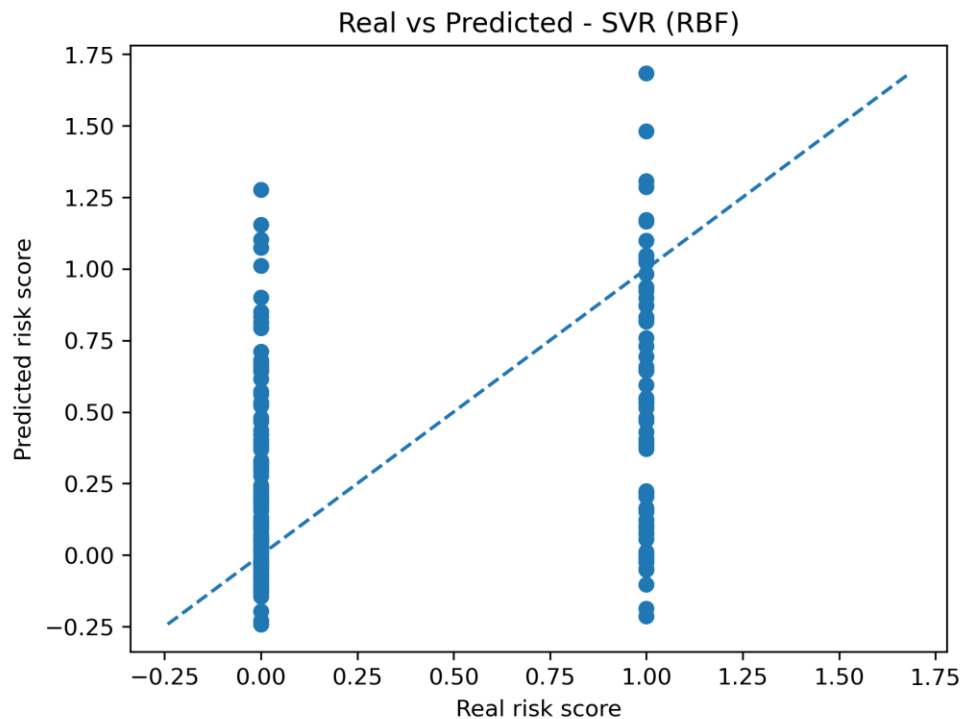


Figure 14. Real vs Predicted - SVR (RBF)

3.5 Model Comparison

Model	MSE	RMSE	R2
Linear Regression	0.007515	0.086687	0.761803
Ridge Regression	0.007530	0.086777	0.761305
SVR (RBF)	0.226275	0.475684	-0.077500

Table 5. Comparison of regression models

From the comparison we observe:

- Linear and Ridge have almost identical results
- SVR has significantly worse generalization
- Linear Regression provides the best balance

3.6 Conclusion from Regression

Linear and Ridge provide stable and accurate risk estimates

Ridge does not provide a significant improvement, indicating little need for regularization

SVR does not provide an improvement and shows unstable generalization

Therefore, for further analysis we use Linear Regression as the final regression approach (with Ridge as confirmation of stability), while SVR is excluded.

The regression results are consistent with the classification and additionally confirm that the selected 13 predictors successfully explain credit risk.

Regression enables:

- finer ranking of clients
- interpretation of predictor influence
- additional validation analysis of the structure of the data

This result is consistent with the ROC-AUC values from classification (≈ 0.74 - 0.75), indicating that the selected predictors have stable, but not extremely strong, discriminatory power.

4. Clustering

4.1 Objective of Clustering

Clustering represents an unsupervised learning approach whose goal is to identify natural segments in the data without using a target variable.

In the context of credit risk, clustering has a practical role:

- segmentation of clients according to similarity
- identification of risky groups
- improvement of credit risk policy
- support for strategic decisions (risk-based pricing, monitoring)

Unlike classification, which predicts default, clustering enables:

- understanding the structure of clients
- discovering hidden groups
- additional validation of risk_score

The target (default_binary) is not used during clustering. The target and risk_score are used only after clustering to interpret the clusters (for example, default rate per cluster).

4.2 Model Selection

The following algorithms were considered in this analysis:

- K-Means
- DBSCAN

Hierarchical clustering was considered theoretically, but due to interpretability, scalability, and practical application, K-Means and DBSCAN were selected as more suitable for credit risk segmentation.

Why K-Means?

- simple and interpretable
- efficient for a medium-sized dataset (~1,000 observations)
- enables clear segmentation and profiling of groups (risk segments)
- suitable for the banking context (policies by segment)
- appropriate because our features are numerical (after encoding), standardized, and we expect groups that are relatively “compact” in feature space

Why DBSCAN?

- detects outliers (noise)
- does not require a predefined number of clusters
- suitable for detecting atypical clients (extreme profiles), which is useful in credit risk (monitoring / fraud / unusual cases)

4.3 Data Preparation

Because K-Means and DBSCAN are distance-based algorithms, the following were performed:

- StandardScaler() standardization
- PCA(2) for visualization (only for graphical presentation)

PCA is used exclusively for visualization, not for the clustering itself. PCA helps us display the clusters in 2D, but clustering is performed in the original standardized feature space.

Although extreme values are observed in the amount variable, they represent real high credit amounts rather than errors in the data. In the basic pipeline, outliers/transformations are not removed, but a controlled experiment with $\log_{1p}(\text{amount})$ was performed in Section 5 to examine sensitivity.

4.4 Selection of an Appropriate Quality Metric

Three metrics were used to determine the optimal number of clusters:

- Elbow Method (inertia / SSE)
- Silhouette Score
- Davies-Bouldin Index

These metrics were selected because they provide a complementary view of quality:

- Elbow (SSE) measures “compactness” (how close the points are to the center of the cluster)
- Silhouette simultaneously measures compactness and separation between clusters
- Davies-Bouldin measures how much the clusters “mix” with each other (lower = better)

Elbow Method

Function:

`KMeans(...).inertia_`

Measures the internal compactness of clusters (SSE).

Lower value -> better compactness.

In practice, we look for the “elbow” where the reduction starts becoming smaller as k increases.

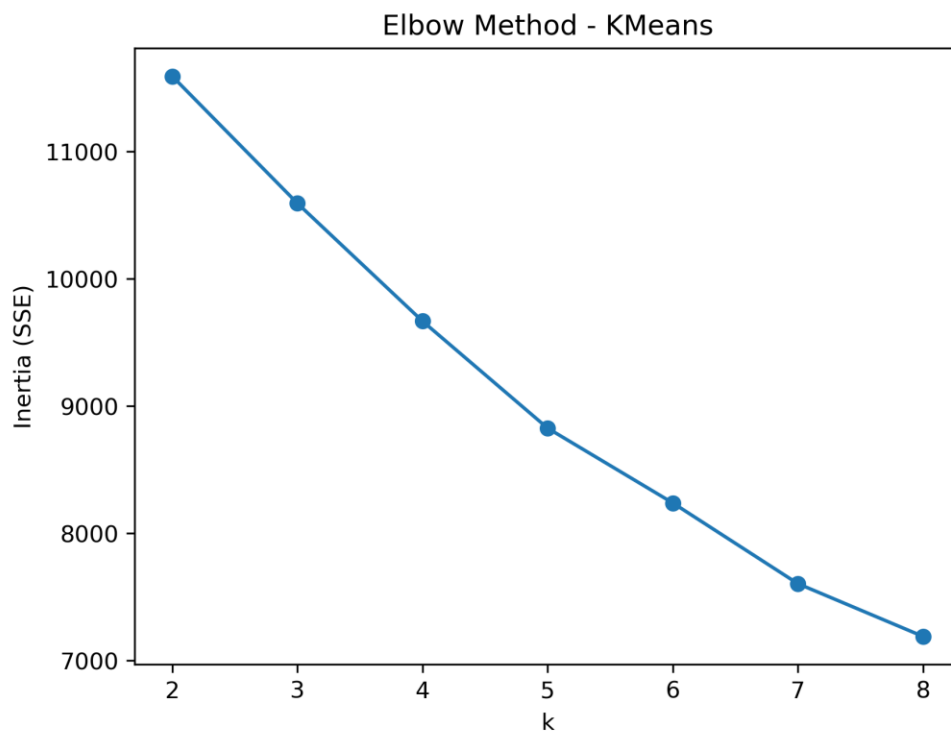


Figure 15: Elbow Method - Kmeans

Интерпретација: графикот покажува дека со зголемување на k , SSE опаѓа, но по одредена точка добивката станува помала. Ова ни дава опсег на разумни k вредности, но не ја дава конечната одлука самостојно.

Silhouette Score

Function:

```
silhouette_score(X_scaled, labels)
```

Value $\in [-1,1]$

- close to 1 -> good separation
- close to 0 -> overlap

In our results, the values are relatively low (~ 0.13 - 0.17), which is expected for real credit data and indicates that the groups partially overlap. Therefore, the selection of k must also include practical interpretation (default rate).

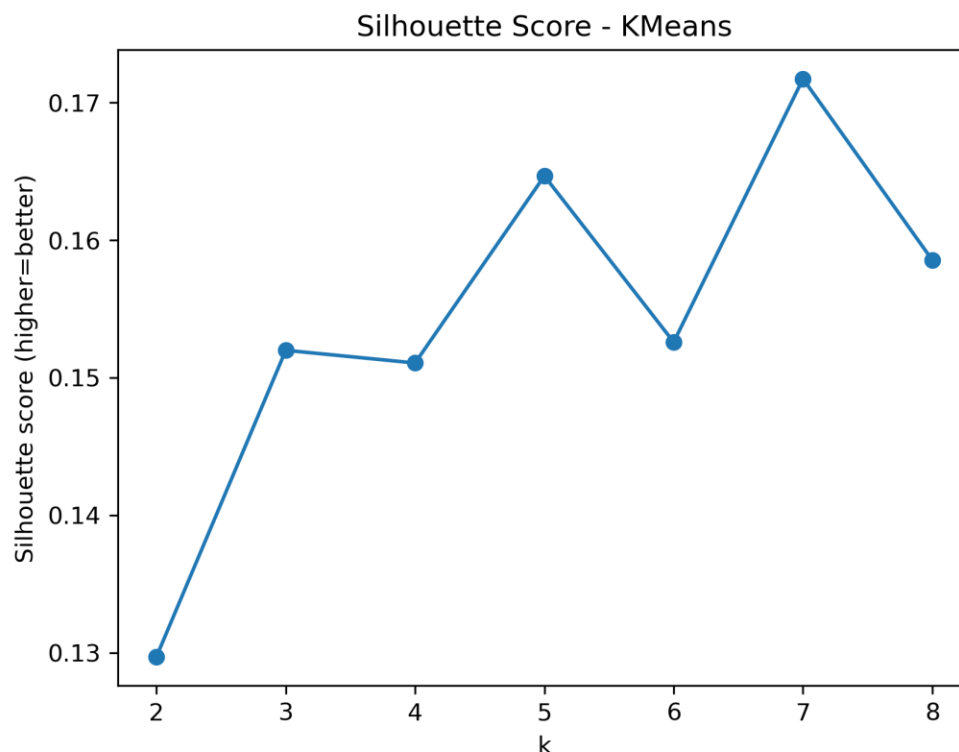


Figure 16: Silhouette Score - Kmeans

Interpretation: the highest silhouette is found at larger k values, but the differences are not dramatic. This means that a larger number of clusters does not necessarily provide a substantially better segmentation, especially if interpretability suffers.

Davies-Bouldin Index

Function:

`davies_bouldin_score(X_scaled, labels)`

- *lower value -> better separation*

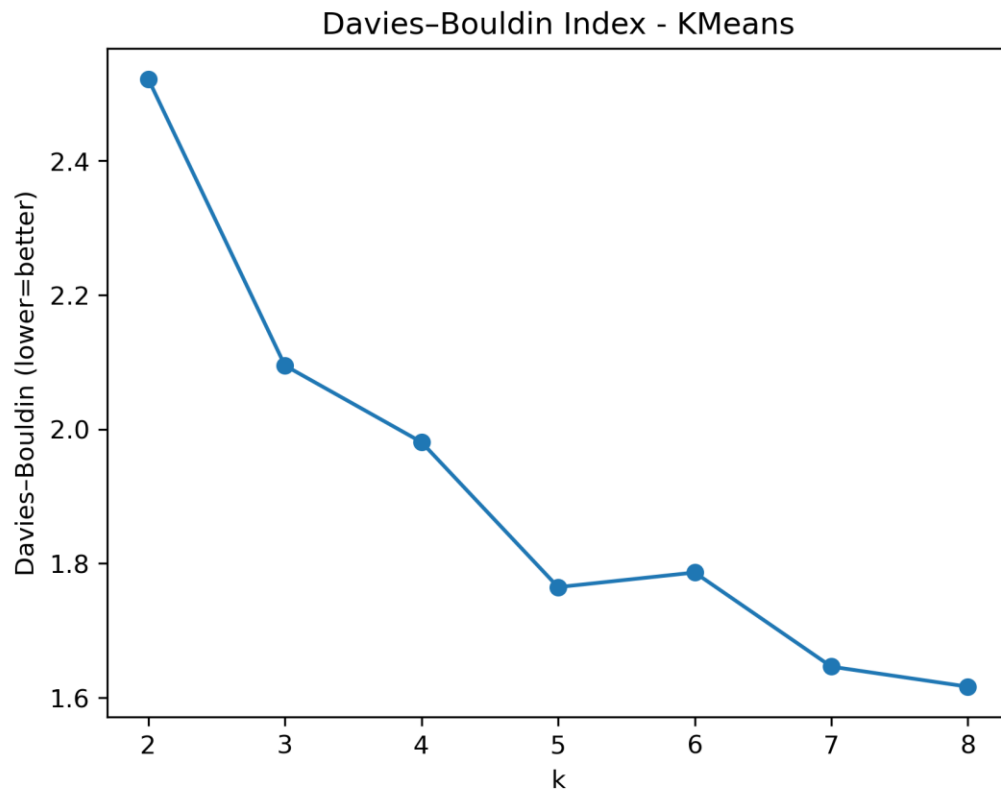


Figure 17: Davies-Bouldin Index - KMeans

Interpretation: DBI generally decreases with larger k, which is normal, but once again the choice must balance metrics + interpretability + differences in risk between clusters.

4.5 Final Selection of k

Although silhouette is highest at $k=7$, practical interpretation and the difference in default rate show that $k=4$ provides the best balance between:

- segmentation
- interpretability
- applicability

Therefore, the final choice was: **KMeans ($k=4$)**

Reason: in a banking context, it is more useful to have 3-5 clear segments (for example: low, medium, medium-high, high risk) than many clusters that are difficult to explain and apply.

4.6 K-Means ($k=4$) - Results

Function::

`KMeans(n_clusters=4, random_state=42, n_init=10)`

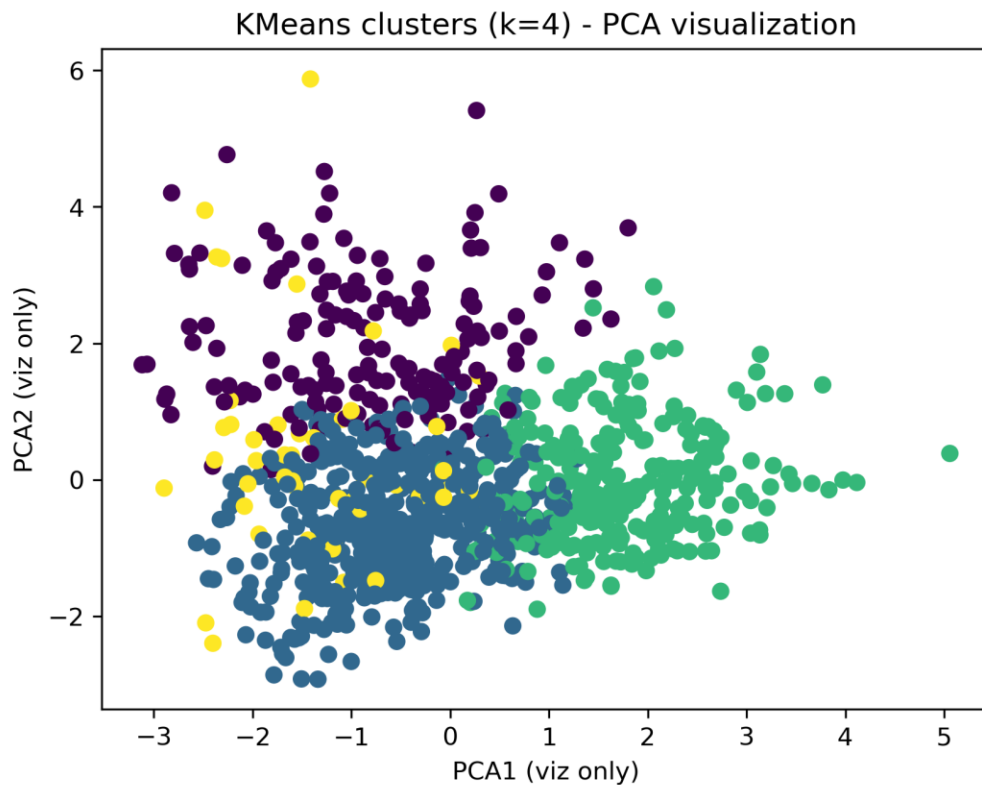


Figure 18: KMeans clusters ($k=4$) - PCA visualization

Interpretation of the PCA visualization: the clusters are visibly grouped, but there is also overlap (which is normal because the silhouette is low). Nevertheless, it can be observed that K-Means manages to capture different “regions” in the space of client profiles, which is sufficient for segment analysis.

4.7 Default Rate by Cluster

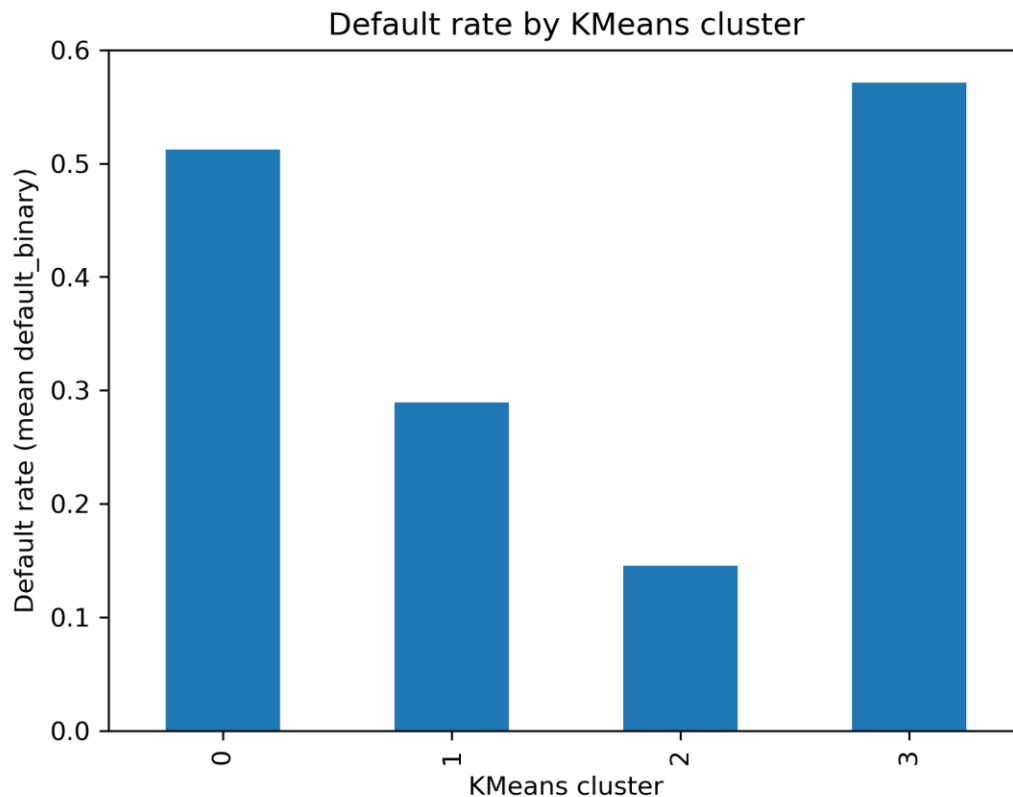


Figure 19: Default rate by KMeans cluster

The clusters show a clear risk gradient:

- Cluster 3 -> ~57% (high risk)
- Cluster 0 -> ~51%
- Cluster 1 -> ~29%
- Cluster 2 -> ~14% (low risk)

This indicates that clustering has practical value: although it is unsupervised, the clusters correspond to actual risk (default). This means that the selected predictors contain enough information to form segments with different default rates.



Figure 20: Mean risk_score by KMeans cluster

Interpretation: the mean risk_score increases in the same order as the default rate, further confirming that risk_score and the clusters are consistent. This is an important result because risk_score is a proxy (predict_proba), yet it still behaves “logically” across segments.

4.8 Cluster Profiles

KMeans cluster profile (mean) displays average values by cluster for the 13 predictors (segment profile).

It serves to interpret the segments (which attributes distinguish high vs. low risk).

It uses mean -> sensitive to outliers, so it is compared with the median.

kmeans_cluster	0	1	2	3
duration	38.09756098	17.09960938	17.41090909	22.69387755
amount	7943.042683	2184.761719	2494.916364	3344.877551
installment_rate	2.591463415	3.001953125	3.12	3.12244898
age	35.48780488	33.19140625	39.83636364	36.26530612
residence_time	2.993902439	2.677734375	3.043636364	2.979591837
existing_credits	1.420731707	1.142578125	1.934545455	1.163265306
people_liable	1.195121951	1.111328125	1.192727273	1.265306122
credit_history_A31	0	0	0	1
credit_history_A34	0.182926829	0.01171875	0.934545455	0
housing_A152	0.548780488	0.73046875	0.8	0.591836735
savings_A64	0.018292683	0.037109375	0.083636364	0.06122449
employment_A72	0.12195122	0.234375	0.083636364	0.183673469
status_A14	0.286585366	0.359375	0.56	0.183673469

Table 6 - KMeans cluster profile (mean)

KMeans cluster profile (median) displays medians by cluster, which are more robust to extreme values (e.g., amount).

It confirms whether the differences between clusters are stable and are not driven by outliers.

It is used as a validation table for the conclusions from the mean profiles.

kmeans_cluster	0	1	2	3
duration	36	15	15	24
amount	7425	1937	2012	2149
installment_rate	2	3	4	4
age	32	30	37	34
residence_time	3	2.5	3	3
existing_credits	1	1	2	1
people_liable	1	1	1	1
credit_history_A31	0	0	0	1
credit_history_A34	0	0	1	0
housing_A152	1	1	1	1
savings_A64	0	0	0	0
employment_A72	0	0	0	0
status_A14	0	0	1	0

Table 7 - KMeans cluster profile (median)

Interpretation:

- High-risk cluster -> on average, it differs through an unfavorable profile (for example: larger amounts/longer durations or weaker indicators in credit_history / status / savings, depending on the specific mean values in the tables).
- Low-risk cluster -> better indicators (better credit history and/or more stable financial characteristics).

This is crucial for the project: the profiles allow us to identify “types” of clients and discuss which predictors most strongly distinguish the segments.

4.9 Comparison of Different Values of k (k=3 and k=5)

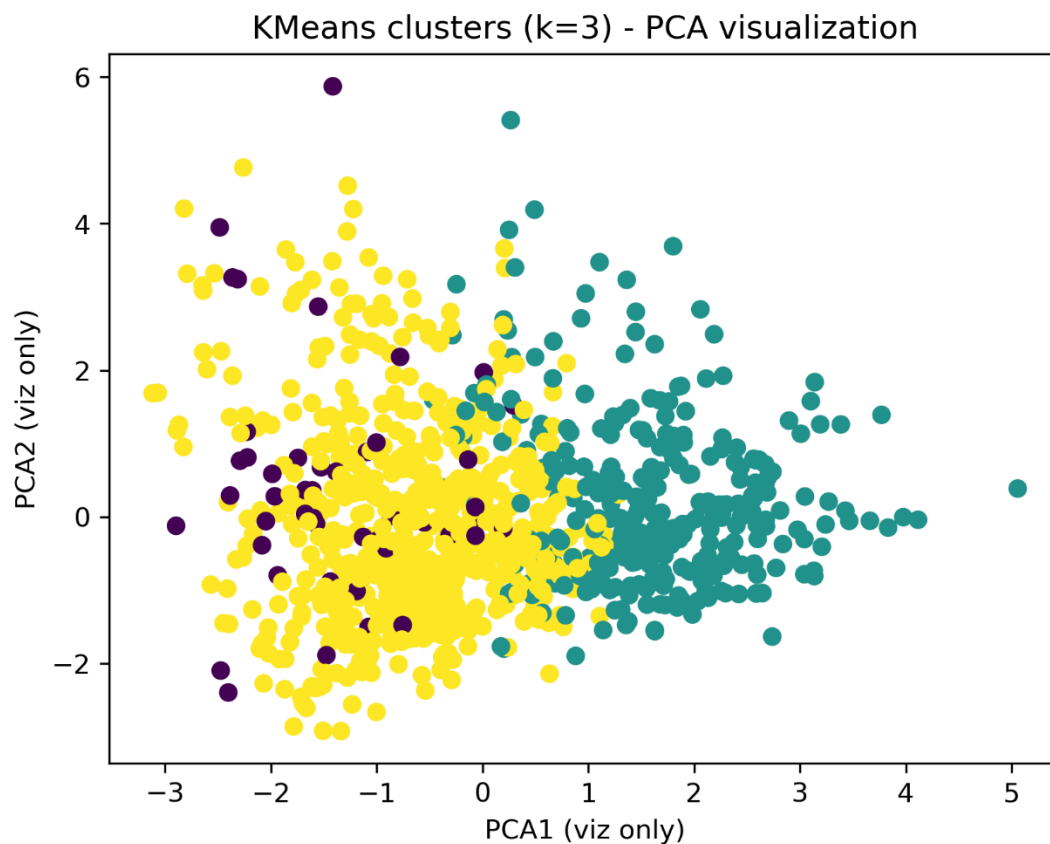


Figure 21. KMeans (k=3) - PCA

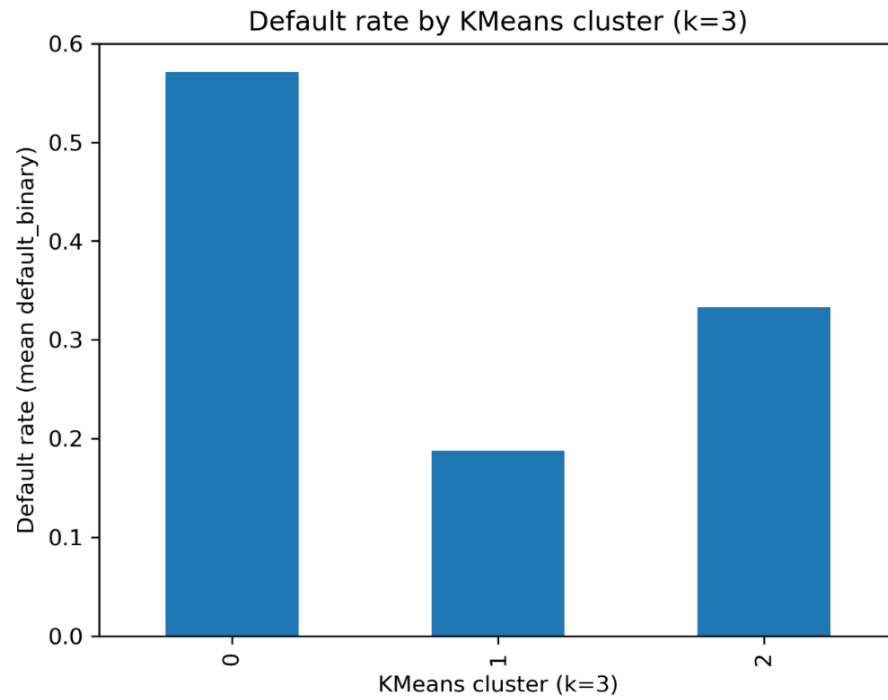


Figure 22. Default rate ($k=3$)

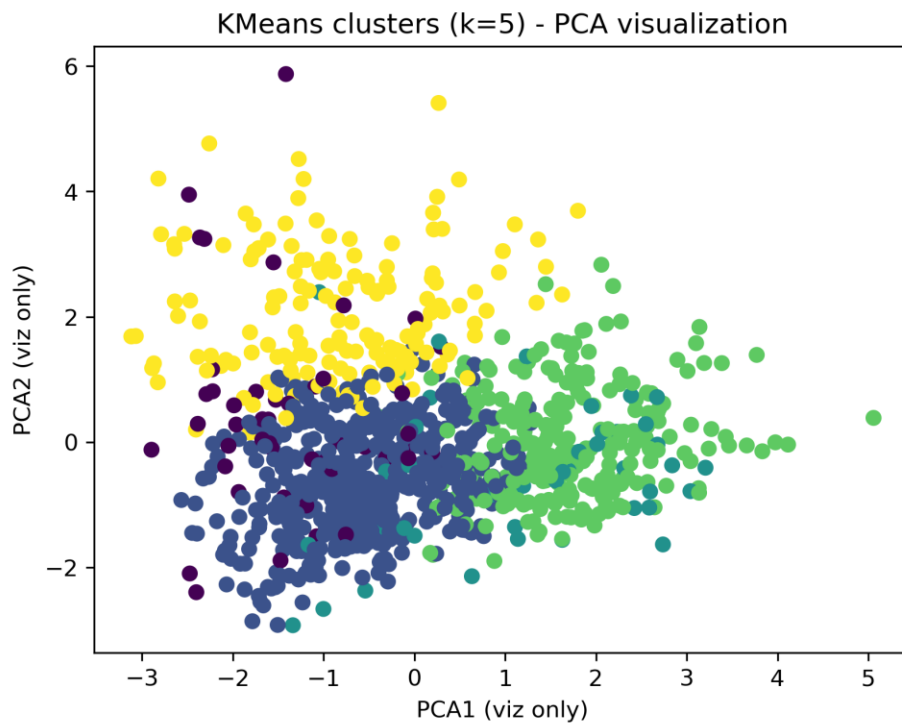


Figure 23. KMeans ($k=5$) - PCA

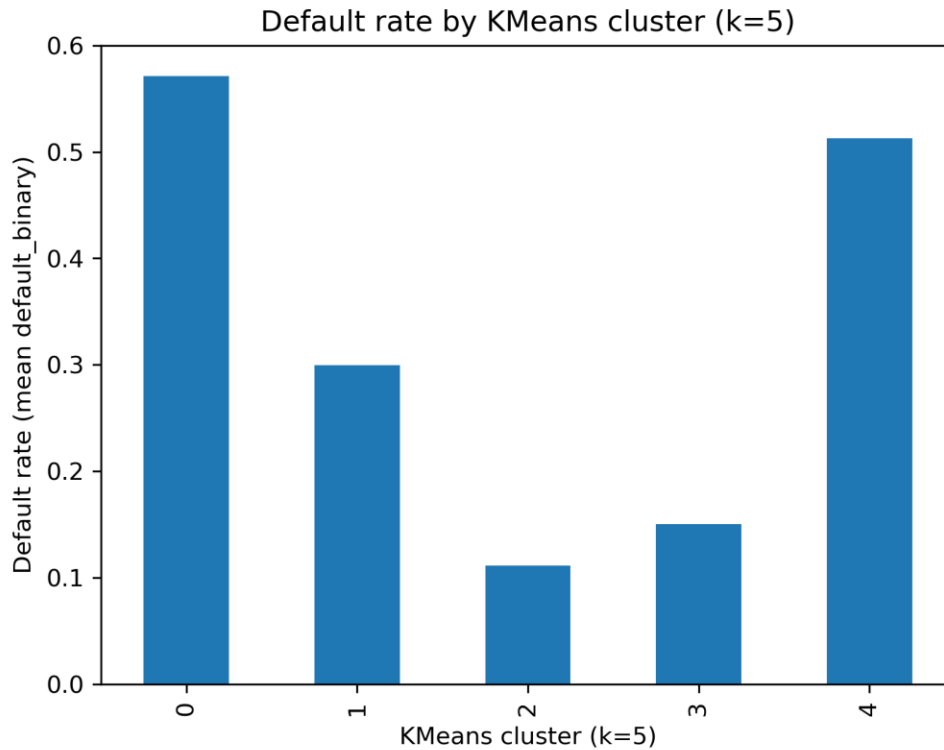


Figure 24. Default rate (k=5)

In order to confirm the choice of the number of clusters, we additionally tested $k=3$ and $k=5$ (in addition to the finally selected model). During evaluation, we did not rely only on metrics (Silhouette, Davies-Bouldin), but also on interpretability and practical risk segmentation according to default rate.

K-Means with $k = 3$

With $k=3$, three segments with different risk levels are obtained (measured through default rate):

- high risk ($\approx 57\%$)
- medium risk ($\approx 33\%$)
- low risk ($\approx 19\%$)

This choice is very easy to communicate (“low/medium/high risk”) and may be sufficient for rough segmentation. However, the segmentation is relatively coarse and does not provide enough detail for policies where additional differentiation in the medium range is required.

The PCA visualization shows that the clusters overlap (as expected according to the silhouette), so $k=3$ mainly serves as a simplified but practical version of the segmentation.

K-Means with k = 5

With k=5, a finer gradient is obtained:

- ≈57% (very high risk)
- ≈51% (high risk)
- ≈30% (medium risk)
- ≈15% (low risk)
- ≈11% (very low risk)

Although this division is more detailed, some clusters have similar risk rates (for example, 57% and 51%), which may reduce the practical need for so many segments while simultaneously making interpretation more complex.

The PCA visualization produces more “sub-segmentation” of the same regions, which does not always mean greater business value.

Final Decision

Although the metrics may suggest larger values of k, the final choice must balance:

- interpretability and applicability in banking logic
- difference in default rate between clusters
- stability of results

Therefore, the choice of a “medium” k (for example, k=4) is the best compromise between detail and clear risk segmentation. With k=4, we obtain sufficiently clear segments with different levels of risk without creating too many groups that are difficult to explain and apply.

4.10 DBSCAN

DBSCAN is included primarily as an outlier/noise analysis because of its sensitivity to density and parameters.

Function:

DBSCAN(eps=0.5, min_samples=5)

Metrics:

- number of clusters
- noise_ratio
- silhouette (non-noise only)

DBSCAN is sensitive to the `eps` and `min_samples` parameters, so a small grid analysis was performed for multiple combinations. This is important because DBSCAN can behave either as a clusterer or as an outlier detector, depending on the density structure of the data.

The combinations (`eps`, `min_samples`) are compared through the number of clusters and `noise_ratio`.

It shows that DBSCAN predominantly labels a large portion of the points as noise, meaning that the structure is not density-based.

This serves as evidence for why DBSCAN is not the main model, but primarily an outlier-checking method.

eps	0.3	0.3	0.5	0.5	0.7	0.7	0.9	0.9	1.1	1.1
min_samples	5	10	5	10	5	10	5	10	5	10
clusters(excl_noise)	0	0	2	0	9	1	17	2	15	5
noise_ratio	1	1	0.989	1	0.922	0.984	0.863	0.969	0.668	0.836
silhouette_non_noise			0.868423811		0.466842011		0.35553273	0.763309132	0.297578716	0.435850895

Table 8 - DBSCAN grid results

Interpretation: for most parameter combinations, DBSCAN labels many points as noise. This indicates that the data does not have clear “density” regions that DBSCAN can stably identify as clusters.

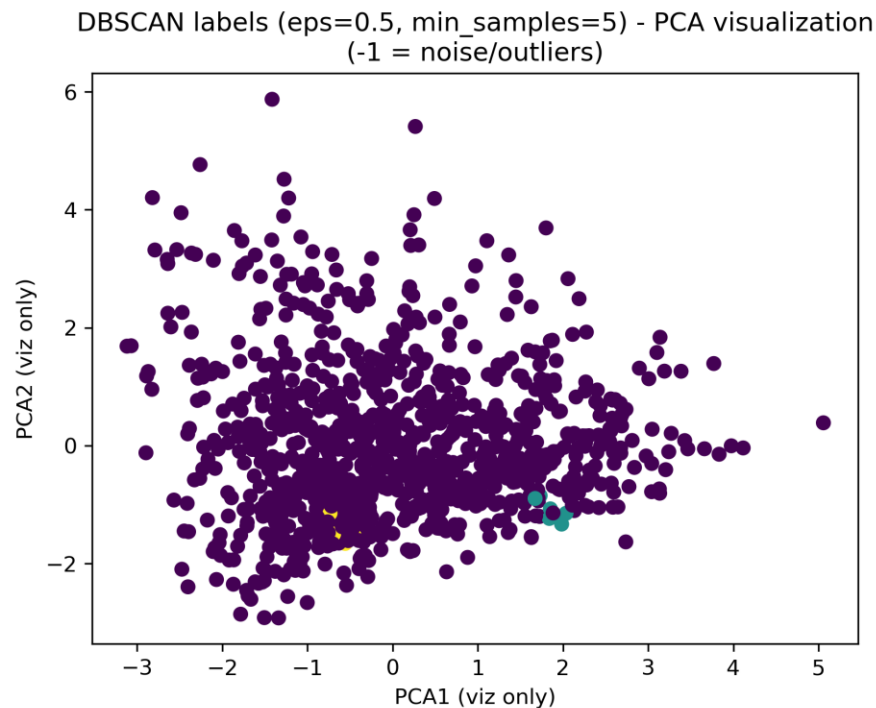


Figure 25: DBSCAN clusters - PCA visualization

Result:

- 2 clusters
- ~98% of points labeled as noise

This indicates that DBSCAN is not suitable for this data structure as the main clustering model because the data does not form a clear density-based structure.

Nevertheless, DBSCAN is useful as an outlier/atypical-client check because these cases may require special attention in credit systems (monitoring, manual review).

4.11 Conclusion from Clustering

- K-Means successfully identifies risky segments and provides useful segmentation according to default rate and risk_score
- k=4 provides the best balance between metrics and interpretability
- DBSCAN did not prove suitable as the main clustering method (due to the high noise_ratio), but it has value as an outlier analysis method
- the clusters are consistent with risk_score and default, confirming that the selected predictors contain structural information about risk

Clustering enables:

- segment-based credit policy (different rules by segment)
- better interpretation of client profiles through cluster profiles (mean/median)
- support for strategic decisions (risk monitoring, ranking, and analysis of risky groups)

4.12 Most Important Predictors and Their Practical Meaning

In the previous phases, 13 predictors were selected based on correlation analysis and financial logic. Their importance is additionally confirmed through:

- stable ROC-AUC value ($\approx 0.74-0.75$)
- high explained variance in regression ($R^2 \approx 0.76$)
- clear segmentation of default rate by cluster

This indicates that these attributes contain substantial structural information about credit risk. The following most important predictors and their practical meaning in the banking context are highlighted:

1. duration (credit duration)

A longer repayment period is associated with a higher probability of default. This is logical because a longer period introduces greater uncertainty and higher accumulated risk.

2. amount (credit amount)

A larger credit amount is positively associated with default risk. Greater indebtedness creates higher financial pressure on the client.

3. installment_rate

A high monthly installment relative to income represents a direct indicator of financial burden and potential risk.

4. credit_history (A31, A34)

Credit history is one of the strongest indicators of future behavior. Clients with unfavorable histories show significantly higher default rates.

5. employment_A72

A short employment period is associated with a less stable financial situation and increased risk.

6. savings_A64

Higher savings are associated with lower risk because they represent a financial reserve in case of unforeseen circumstances.

7. housing_A152 (own housing)

Owning a home is an indicator of financial stability and lower default risk.

8. age

Older clients generally show lower risk, which may be associated with financial maturity and stability.

Practical Meaning

These results confirm that credit risk is not random, but is structurally associated with:

- financial burden
- income stability
- credit history
- accumulated assets

In banking applications, these factors can be directly used for:

- automated risk assessment
- risk-based pricing
- defining different credit policies by segment
- early risk detection

This closes the analytical cycle: from detecting statistically significant predictors, through modeling, to practical interpretation.

5. Additional Techniques and Validation Experiments

This project included additional techniques with the goal of improving the practical applicability of the models and addressing typical problems in credit risk data (imbalanced classes, increased dimensionality after encoding, threshold sensitivity). The techniques were applied as controlled steps without disrupting the basic pipeline.

5.1 Handling Imbalanced Classes (Cost-Sensitive Learning)

Since approximately 30% of the samples are “bad” clients, a cost-sensitive approach was applied through `class_weight='balanced'`, and **threshold tuning** was additionally performed.

This is justified because in a banking context it is more critical to miss a risky client (false negative) than to perform an additional check on a low-risk client (false positive).

The results showed that selecting a lower threshold can significantly increase recall for the risky class, which is practically more important than maximizing accuracy.

5.2 Dimensionality Reduction and Complexity Control

Due to one-hot encoding, the number of attributes increased to approximately 50, which can increase complexity and the risk of overfitting.

Therefore, **feature selection** was applied and the models were trained using the **13 most important predictors**, improving interpretability and reducing complexity.

Additionally, **Ridge regression** was used as a stability check, while **PCA(2)** was used exclusively for visualization in clustering (clustering itself is performed in the original standardized feature space).

5.3 Feature Engineering Experiments (Controlled Improvements)

Two controlled feature engineering experiments were performed in order to determine whether simple transformations provide better discriminatory power and better risk segmentation without changing the basic pipeline.

5.3.1 Experiment 1

log1p(amount) instead of amount

Because the amount variable has a right-skewed distribution and pronounced extreme values, the following transformation was applied:

$$\text{amount_log1p} = \log(1 + \text{amount})$$

Results (experiment):

- PR-AUC (AP) ≈ 0.617
- ROC-AUC ≈ 0.768
- Recall_{bad} at threshold 0.50 ≈ 0.433

For clustering (K-Means), the default rate by cluster showed a clear gradient ($\approx 0.58, 0.34, 0.19, 0.11$), with separation (max-min) ≈ 0.47 .

Interpretation: the log transformation produced a moderate improvement in ROC-AUC and better segmentation by clusters, but recall for the risky class remained approximately the same.

5.3.2 Experiment 2

Interaction feature - average_monthly_principal = amount/duration

In order to construct a feature that represents the approximate monthly principal component of the credit, the following variable was created:

$$\text{average_monthly_principal} = \text{amount/duration}$$

This represents the average amount of principal allocated to each month of the credit duration. It should not be interpreted as the actual monthly installment, because the dataset does not provide the complete repayment schedule or interest component.

This represents the average monthly principal amount of the credit.

Results (experiment):

- PR-AUC (AP) ≈ 0.595
- ROC-AUC ≈ 0.766
- Recall_{bad} at threshold 0.50 ≈ 0.500

For clustering, the default rate by cluster was ($\approx 0.51, 0.32, 0.15, 0.125$), with separation (max-min) ≈ 0.38 .

Interpretation: the interaction feature improved recall (0.50 instead of ~ 0.43), meaning better detection of risky clients, but the segmentation by clusters was somewhat weaker compared with $\log_{1p}(\text{amount})$.

5.3.3 Conclusion from the Experiments

Although neither transformation produced a dramatic improvement, the results show that:

- `amount` has a nonlinear structure that is partially improved through log transformation
- `monthly_burden` has a clear banking interpretation and positively affects recall
- the basic model with the selected 13 predictors already provides stable discriminatory power

Therefore, the basic pipeline remains the final pipeline, while the experiments serve as validation analysis for stability and possible directions for future improvements.

5.4 Note on Additional Ensemble Methods

The possibility of applying more advanced ensemble methods (for example, Gradient Boosting) was considered.

However, given the moderate size of the dataset (1,000 instances) and the already stable performance ($\text{ROC-AUC} \approx 0.75$), further increasing complexity was not expected to bring a significant improvement in generalization.

Therefore, the focus was placed on interpretability and cost-sensitive optimization, which are more critical in a banking context.

6. Conclusion

In conclusion, this project confirms the possibility of successfully modeling and interpreting credit risk through a combination of analytical techniques and machine learning models.

The initial goal was to build a model that would not only predict whether a client represents a risk of default, but would also enable a deeper understanding of the factors that influence credit risk and their practical application in a banking context.

The results show that the basic models provide moderate discriminatory power (ROC-AUC $\approx$ 0.75), but their ability to detect risky clients is limited.

Through the application of a cost-sensitive approach (`class_weight='balanced'`) and threshold selection (threshold tuning), recall for the risky class is significantly improved, which is crucial in banking conditions where missing a risky client represents a greater loss than performing an additional check.

Regression analysis confirms that `risk_score` can be stably modeled using a linear approach, enabling client ranking according to risk and additional interpretation of predictor influence.

At the same time, clustering identifies clear client segments with different default rates, showing that the data contains structural patterns that can be used for segment analysis and decision-making.

The predictor analysis shows that credit risk is most strongly associated with financial burden (amount, duration, `installment_rate`), client stability (employment, savings, housing), and credit history.

These factors have a clear economic interpretation and directly influence the probability of default.

Therefore, the project not only provides prediction models, but also extracts significant insights that can be applied in real banking systems, such as:

- segment-based credit policy
- risk-based pricing
- early warning systems
- prioritization of clients for additional review

Finally, it can be concluded that the greatest value does not come only from the accuracy of the models, but from their interpretability and ability to support informed and practical decision-making in credit risk management.

References:

1. Breiman, L. (2001). *Random forests*. Machine Learning, 45, 5–32. <https://doi.org/10.1023/A:1010933404324>
2. Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20, 273–297. <https://doi.org/10.1007/BF00994018>
3. Davies, D. L., & Bouldin, D. W. (1979). *A cluster separation measure*. IEEE Transactions on Pattern Analysis and Machine Intelligence, PAMI-1(2), 224–227. <https://doi.org/10.1109/TPAMI.1979.4766909>
4. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). *A density-based algorithm for discovering clusters in large spatial databases with noise*. In Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96) (pp. 226–231).
5. Fawcett, T. (2006). *An introduction to ROC analysis*. Pattern Recognition Letters, 27(8), 861–874. <https://doi.org/10.1016/j.patrec.2005.10.010>
6. Feng, C., Wang, H., Lu, N., Chen, T., He, H., Lu, Y., & Tu, X. M. (2014). *Log-transformation and its implications for data analysis*. Shanghai Archives of Psychiatry, 26(2), 105–109. <https://doi.org/10.3969/j.issn.1002-0829.2014.02.009>
7. Guyon, I., & Elisseeff, A. (2003). *An introduction to variable and feature selection*. Journal of Machine Learning Research, 3, 1157–1182.
8. He, H., & Garcia, E. A. (2009). *Learning from imbalanced data*. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263–1284. <https://doi.org/10.1109/TKDE.2008.239>
9. Hofmann, H. (1994). *Statlog (German Credit Data) [Dataset]*. UCI Machine Learning Repository. <https://doi.org/10.24432/C5NC77>
10. Hodge, V. J., & Austin, J. (2004). *A survey of outlier detection methodologies*. Artificial Intelligence Review, 22(2), 85–126. <https://doi.org/10.1023/B:AIRE.0000045502.10941.A9>
11. Louzada, F., Ara, A., & Fernandes, G. B. (2016). *Classification methods applied to credit scoring: Systematic review and overall comparison*. Surveys in Operations Research and Management Science, 21(2), 117–134. <https://doi.org/10.1016/j.sorms.2016.10.001>
12. Rousseeuw, P. J. (1987). *Silhouettes: A graphical aid to the interpretation and validation of cluster analysis*. Journal of Computational and Applied Mathematics, 20, 53–65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
13. Saito, T., & Rehmsmeier, M. (2015). *The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets*. PLOS ONE, 10(3), e0118432. <https://doi.org/10.1371/journal.pone.0118432>
14. Thomas, L. C., Edelman, D. B., & Crook, J. N. (2002). *Credit Scoring and Its Applications*. Society for Industrial and Applied Mathematics (SIAM).
15. Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.